Master's Thesis

AI Usage in CI/CD/CT Pipelines for Compute Platforms in Automotive

by

Morris Darren Babu

Matrikel-Nr.: 23206441

Supervision:
University Supervisor:
Prof. Dr.-Ing. Jürgen Teich
Dr.-Ing. Stefan Wildermann

Industrial Supervisor (CARIAD SE):
Dipl.-Ing. (FH) Bernd Schaller
Dr. Andreas Weichslgartner

March 6, 2026

This document was produced with the typesetting system LaTeX2e.

Master Thesis Objectives

Description:

Modern automotive software systems exceed 100 million lines of code per vehicle, creating significant challenges for security testing at scale. Fuzz testing is an effective technique for discovering software vulnerabilities, but writing fuzz drivers requires deep API knowledge and considerable manual effort. This thesis investigates whether Large Language Models (LLMs) can automate fuzz driver generation for C/C++ automotive libraries and whether the resulting approach is viable for enterprise CI/CD pipelines.

The following tasks shall be addressed:

1. **Literature Review:** Survey the state of the art in LLM-based code generation, fuzz testing techniques, and CI/CD pipeline security integration. Identify research gaps for automotive applications.

2. **Framework Design:** Design a technical framework for automated LLM-based fuzz driver generation, comprising automated API context extraction, prompt construction, and coverage-based evaluation.

3. **Empirical Evaluation:** Systematically evaluate multiple open-source LLMs (varying in size, architecture, and specialization) on their ability to generate compilable, executable, and coverage-achieving fuzz drivers for C++ libraries.

4. **Model Optimization:** Investigate whether domain-specific fine-tuning (LoRA) of smaller models can match or exceed larger general-purpose models in fuzz driver quality and efficiency.

5. **Enterprise Integration:** Deploy and validate the approach within a real automotive enterprise CI/CD environment (CARIAD SE), addressing network isolation, security compliance, and cost constraints.

6. **Analysis and Discussion:** Analyze the results with respect to effectiveness, optimization potential, and enterprise feasibility. Discuss limitations, implications for practice, and future research directions.

Thesis Period: May 2025 – September 2025

Abstract

Modern automotive software exceeds 100 million lines of code per vehicle, making security testing essential under ISO 26262 and UNECE WP.29. Fuzz testing effectively discovers vulnerabilities, but writing fuzz drivers requires deep API knowledge that many teams lack. This thesis investigates whether Large Language Models (LLMs) can automate fuzz driver generation for C/C++ libraries and whether the approach is viable for enterprise CI/CD/CT pipelines.

We evaluated 14 LLMs (1.5B–46.7B parameters) on `yaml-cpp` and additional C++ targets. The results were counterintuitive: larger models did not perform better. General-purpose models such as Yi (34B) produced non-functional drivers, and Mixtral (46.7B) could not be evaluated due to hardware constraints. In contrast, Gemma 3 (27B) achieved 45.06% line coverage and Qwen 2.5-Coder (32B) reached 43.08%. Several code-specialized models including CodeLlama (34B) also failed, showing that neither size nor specialization label alone predicts success.

Fine-tuning a compact 1.5B model with Low-Rank Adaptation (LoRA) on OSS-Fuzz drivers reduced generation time by 33% and token usage by 55% while maintaining quality. Enterprise deployment at CARIAD SE revealed that infrastructure challenges, such as corporate firewalls blocking API calls from isolated CI runners, required architectural solutions (Azure Private Link, self-hosted runners).

Overall, LLM-based fuzz driver generation is technically viable for automotive security testing when models are chosen carefully and infrastructure is adapted to enterprise constraints.

Keywords: Large Language Models, Fuzz Testing, Automotive Security, CI/CD/CT Integration, LoRA Fine-Tuning, ISO 26262, Code Coverage.

Contents

List of Figures xiii

List of Tables xv

List of Abbreviations xvii

1 Introduction 1
- 1.1 Background and Motivation . . . 1
  - 1.1.1 Security Challenges in Modern Systems . . . 2
  - 1.1.2 Fuzzing . . . 2
  - 1.1.3 CI/CD/CT Pipeline Context . . . 4
- 1.2 Problem Statement . . . 4
- 1.3 Research Questions and Objectives . . . 6
  - 1.3.1 Primary Research Questions . . . 6
- 1.4 Thesis Organization . . . 7

2 Literature Review 9
- 2.1 Traditional AI Approaches in Software Testing . . . 9
- 2.2 Large Language Models: Evolution and Code Generation . . . 10
  - 2.2.1 Low-Rank Adaptation (LoRA) . . . 11
- 2.3 Foundations of AI-Guided Fuzzing Techniques . . . 11
- 2.4 AI-Enhanced Fuzzing: State of the Art . . . 12
  - 2.4.1 Research Gap of LLM-Based Fuzzing Techniques . . . 13
- 2.5 CI/CD/CT Pipeline Security Integration . . . 14
- 2.6 Automotive Software Development and Safety Standards . . . 14
- 2.7 Research Gaps and Thesis Positioning . . . 15

3 Methodology and Framework 17
- 3.1 Conceptual Framework: AI-Driven White-Box Fuzzing . . . 17
  - 3.1.1 The Fuzz Driver Problem . . . 17
  - 3.1.2 Our Approach: Automated Generation . . . 18
- 3.2 Technical Architecture Design . . . 18
  - 3.2.1 Input Layer (Automated Context Extraction) . . . 20
  - 3.2.2 Generation Layer (LLM Interface) . . . 20
  - 3.2.3 Execution Layer (Evaluation Pipeline) . . . 20

3.3 Research Strategy: Three-Phase Approach 21
3.3.1 Phase 1: Local LLM Evaluation 21
3.3.2 Phase 2: Model Optimization with LoRA 21
3.3.3 Phase 3: Enterprise CI/CD/CT Integration 22
3.4 Evaluation Methodology 22

4 Implementation **25**
4.1 Toolchain Selection and Rationale 25
4.1.1 Fuzzing Infrastructure 25
4.1.2 LLM Infrastructure 26
4.1.3 Development Environment 27
4.2 Phase 1: Local LLM Evaluation Setup 28
4.2.1 Benchmarked Models 28
4.2.2 Target Repositories 29
4.2.3 Evaluation Environment 30
4.3 Phase 2: Model Optimization with LoRA Fine-Tuning 32
4.3.1 Training Data Preparation 32
4.3.2 LoRA Configuration and Training 33
4.3.3 Fine-Tuned Model Evaluation 34
4.4 Phase 3: Enterprise CI/CD/CT Integration 35
4.4.1 Architectural Challenges 36
4.4.2 Self-Hosted Runner Solution 38
4.4.3 Operational Considerations 41

5 Experimental Results **43**
5.1 Experimental Setup 43
5.1.1 Target Selection and Criteria 43
5.1.2 Hardware and Software Configuration 44
5.2 LLM Fuzz Driver Generation Results 44
5.2.1 Successful Models: Performance Data 44
5.2.2 Unsuccessful Models: Critical Findings 45
5.2.3 Cross-Library Validation 47
5.3 Model Optimization Results 49
5.3.1 LoRA Fine-Tuning Efficiency 49
5.3.2 Comparative Analysis 50
5.4 Economic Analysis and Resource Metrics 51
5.4.1 Azure OpenAI Cost Projections 51
5.4.2 Comparison with Manual Testing 51
5.5 Summary 52

6 Discussion and Conclusion **55**
6.1 Interpretation of Results 55
6.1.1 What the Data Reveals 55
6.1.2 Unexpected Findings: Network Architecture 56
6.2 Addressing Research Questions 58
6.2.1 Primary Research Question Analysis 58
6.2.2 Secondary Research Questions Analysis 60
6.3 Limitations and Constraints 61
6.3.1 Technical Limitations 61
6.3.2 Methodological Boundaries 61
6.4 Implications for Practice 62
6.4.1 Automotive Industry Impact 62
6.4.2 Enterprise CI/CD/CT Deployment Patterns 62
6.5 Future Research Directions 63
6.6 Contributions 64
6.7 Conclusion 64

A Appendix **67**
A.1 Evaluated Model Summary 67
A.2 LoRA Fine-Tuning Script 67
A.3 Sample Generated Fuzz Driver 68
A.4 Pipeline Configuration 69
A.5 Sample LLM Prompt for Fuzz Driver Generation 71

References **73**

List of Figures

1.1 The Fuzzing Process. Data artifacts are shown in rectangular boxes; processing steps are shown in rounded boxes. The manual creation of fuzz drivers (highlighted in red) is the bottleneck this thesis aims to automate. 3

1.2 The Automotive CI/CD/CT Pipeline. Data artifacts are shown in rectangular boxes; processing steps are shown in rounded boxes. Security validation (highlighted) is where fuzzing integrates into the pipeline. The feedback loop from telemetry to the source tree represents the continuous improvement cycle. 5

2.1 The V-Model Development Lifecycle (adapted from ISO 26262 [22]). The diagram illustrates the relationship between each development phase (left) and its corresponding testing phase (right), converging at the Code implementation. 15

3.1 Technical Architecture of our LLM-based fuzz driver generation pipeline. The three layers are marked with dashed borders. Data artifacts are shown in rectangular boxes; processing steps in boxes with rounded corners. Green/yellow highlighted elements denote new contributions of this work with respect to the state of the art (cf. Figure 1.1). 19

3.2 Enterprise Integration Strategy. The self-hosted CI/CD/CT runner communicates with Azure OpenAI through a Private Link endpoint, routing traffic through the corporate firewall over a private connection without traversing the public internet. 22

4.1 Evaluation Environment Component Diagram. Components (rounded boxes) interact via defined interfaces. Ollama or llama.cpp serve the LLM; cifuzz spark orchestrates driver generation; CMake/Clang builds the driver; libFuzzer executes it; llvm-cov measures coverage. All build and fuzzing components run inside a Podman container. The top-down flow shows the data transformation pipeline from source code to coverage report. 31

4.2 CI/CD/CT Pipeline Workflow. Processing steps are in rounded boxes; data artifacts in rectangular boxes. The highlighted step (cifuzz spark) invokes the LLM via Azure Private Link. Corpus persistence through JFrog Artifactory enables cumulative coverage improvement across runs. 40

5.1 Code Coverage by Model (yaml-cpp). Only three of the fourteen evaluated models produced functional fuzz drivers. 46

6.1 Enterprise Network Architecture with Azure Private Link. Processing components are shown in rounded boxes; data stores in rectangular boxes. All edges are labeled with the data exchanged between components. Traffic flows through a private connection and never traverses the public internet. 57

List of Tables

4.1 Summary of All Evaluated Models on yaml-cpp 29
4.2 Fine-Tuned Model Evaluation Results on yaml-cpp (Absolute Numbers) 35
4.3 Azure OpenAI Cost Projections for Different Usage Scenarios . . . 41

5.1 Model Performance Metrics on yaml-cpp Repository 44
5.2 Unsuccessful Models and Failure Analysis on yaml-cpp 46
5.3 Model Performance on fmt Library 47
5.4 Model Performance on pugixml Library 48
5.5 Qwen 2.5-Coder 32B Performance Across Additional Libraries . . . 49
5.6 LoRA Fine-Tuning Efficiency Improvements 50
5.7 Azure OpenAI Cost Projections for Different Usage Levels 51

A.1 Complete List of Evaluated Models 67

List of Abbreviations

ABS	Anti-lock Braking System
ADAS	Advanced Driver-Assistance Systems
AFL	American Fuzzy Lop
AI	Artificial Intelligence
API	Application Programming Interface
ASan	AddressSanitizer
AST	Abstract Syntax Tree
AUTOSAR	AUTomotive Open System ARchitecture
BRS	Business Requirements Specification
CD	Continuous Delivery / Continuous Deployment
CI	Continuous Integration
CT	Continuous Testing
DNS	Domain Name System
ECU	Electronic Control Unit
GPU	Graphics Processing Unit
HLD	High-Level Design
JSON	JavaScript Object Notation
LLD	Low-Level Design
LLM	Large Language Model
LLVM	Low Level Virtual Machine
LoRA	Low-Rank Adaptation

MTTV	Mean Time To Vulnerability
SAE	Society of Automotive Engineers
SRS	System Requirements Specification
UBSan	UndefinedBehaviorSanitizer
UNECE	United Nations Economic Commission for Europe
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language

1 Introduction

1.1 Background and Motivation

Automotive software has changed fundamentally in the last 15 years. Industry reports indicate that a modern car runs over 100 million lines of code, distributed across more than 100 Electronic Control Units (ECUs) [7, 5]. This scale illustrates a complete shift in how vehicles are engineered.

This shift came from somewhere. Vehicles used to be mechanical systems. An engine controller managed fuel injection. An ABS system prevented wheel lockup. These systems mostly operated independently. A skilled engineer could understand the complete electronic architecture within weeks. That is gone now.

Today, vehicles are software-defined platforms. Tesla demonstrated this most clearly with over-the-air updates that deliver new features, improved performance, and safety enhancements without requiring a service visit [26]. Traditional automakers observed this shift and recognized that software now determines competitive advantage, from acceleration performance to customer-facing features. Mechanical engineering alone no longer drives differentiation.

This transformation created a problem. Software is complex. A single vehicle contains code spanning multiple programming paradigms, different safety criticality levels, and various execution environments. Code that is safety-critical on AUTOSAR [2] platforms follows entirely different development rules than infotainment applications running on Linux. Real-time control systems operating at microsecond intervals coexist with cloud-connected services. This diversity makes testing exponentially harder.

Consider a Level 2 automated driving system. Perception algorithms process camera, radar, and lidar data. Planning modules compute safe trajectories. Control systems translate trajectories into steering and throttle commands. Each layer requires different technical expertise: computer vision, machine learning, control theory, real-time systems. Integration alone consumes thousands of person-hours per release cycle. When legacy code is added to this, code written years ago with minimal documentation, understanding what the software actually does becomes detective work.

1.1.1 Security Challenges in Modern Systems

This complexity creates security risks that are genuinely dangerous. When a web server gets compromised, the consequences are serious. Data theft, service disruption, financial loss. When a vehicle control system is compromised, it can result in severe physical harm or fatal injuries. This is not theoretical. In 2015, researchers demonstrated remote exploitation of a Jeep Cherokee. They showed scenarios where attackers could disable brakes or seize steering at highway speeds [31]. The resulting recall affected 1.4 million vehicles [18]. The infotainment system, connected via cellular networks, had provided a pathway to safety-critical networks.

Regulators noticed a critical security gap in vehicle software. UNECE Regulation 155 now requires cybersecurity management systems for all new vehicle types [39]. ISO/SAE 21434 sets cybersecurity engineering requirements across the entire vehicle lifecycle [23]. These are not suggestions. Vehicles that do not comply cannot be sold in regulated markets. Manufacturers must demonstrate that security was designed into systems from the beginning, not bolted on afterward.

This regulatory pressure collides with a fundamental tension in automotive security testing. Safety-critical systems undergo exhaustive verification against precisely defined requirements, like ISO 26262 [22]. Every possible failure mode is analyzed. Fixes are tested. This works for safety, where hazards are known and physics are deterministic.

Security is different. The adversary is intelligent, creative, and unconstrained by specifications. Attackers do not limit themselves to documented interfaces. They probe for implementation weaknesses, logic errors, overlooked edge cases. Testing for security means exploring the unexpected, not verifying the expected. This requires fundamentally different approaches.

1.1.2 Fuzzing

The fuzzing architecture begins with the human element. The developer writes the initial source code and possesses the critical domain knowledge required to understand the intended software logic. Because an automated system cannot inherently know what specific areas to target, the developer must manually define the testing scope and create the fuzz driver, a small program that translates raw fuzzer inputs into meaningful API calls. This manual creation step typically requires 2 to 8 hours of specialized engineering time per target library [12], representing the primary bottleneck that this research aims to automate.

Once the fuzz driver and seed inputs are supplied, the automated fuzzing engine takes control. As illustrated in Figure 1.1, the engine continuously executes the fuzz driver against the target software, feeding it generated inputs while actively monitoring for unexpected behavior. If a crash is detected, the engine extracts the crash data, and a vulnerability report is generated. Conversely, if the execution completes

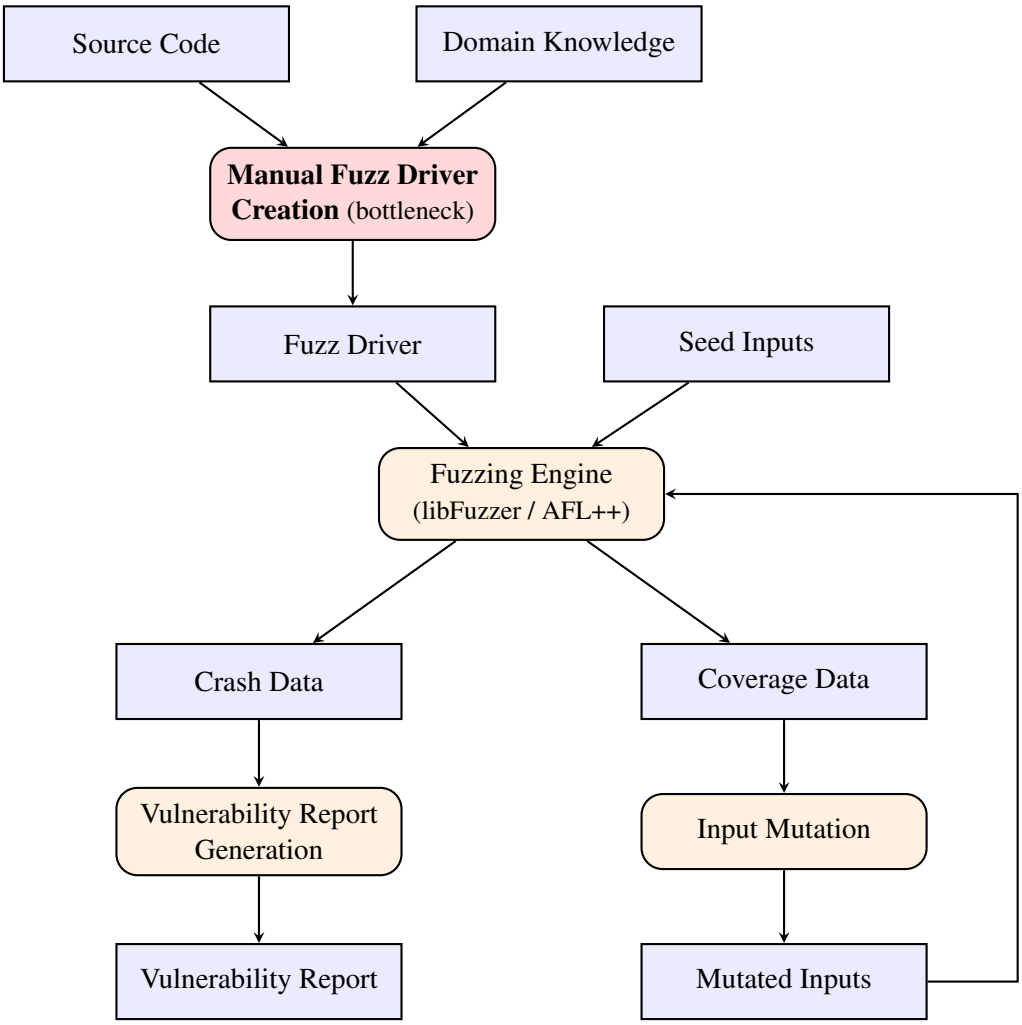

Figure 1.1: The Fuzzing Process. Data artifacts are shown in rectangular boxes; processing steps are shown in rounded boxes. The manual creation of fuzz drivers (highlighted in red) is the bottleneck this thesis aims to automate.

without error, the engine collects coverage data to understand which code paths were reached. An input mutation step then creates slightly altered variations of the previous inputs. These mutated inputs are fed back into the execution loop, driving a continuous cycle of automated vulnerability discovery.

1.1.3 CI/CD/CT Pipeline Context

Modern software development has embraced a philosophy: integrate code frequently, test automatically, deploy rapidly. The reasoning is sound. Small, incremental changes are easier to understand and debug than large infrequent releases. Automated testing catches regressions before they spread. Fast feedback loops let developers address problems while context is fresh [21, 14].

The automotive industry has adopted these principles, adapting them to domain-specific constraints. Untested code cannot be casually pushed to vehicles traveling at highway speeds. Automotive continuous integration pipelines typically feed into formal release processes with human review and additional validation before software reaches production vehicles.

The core principles remain: automate where possible, test early and often, and maintain a continuously releasable codebase [25].

This creates a tension. A well-optimized pipeline might complete build and basic testing in 15 to 30 minutes. Developers expect rapid feedback. But security testing, especially fuzzing, often requires extended execution. A fuzzing campaign might run for hours or days to discover subtle vulnerabilities [12]. This timescale is incompatible with developer expectations for fast feedback.

Enterprise environments add additional complexity. Large organizations balance computational demands against security requirements. Build runners may execute in public cloud for scalability while sensitive operations stay in private networks isolated from external connections [3]. Network policies restrict communication between zones. A simple task like calling an LLM API from a build runner encounters firewalls, proxy servers, and access control systems.

1.2 Problem Statement

The core problem is straightforward: code production vastly outpaces security analysis capacity.

In the broader automotive software industry, massive development teams deploy continuous code modifications. For example, at CARIAD, hundreds of developers produce thousands of changes daily. Even well-staffed security teams cannot review each change for vulnerabilities. Automated tools like SonarQube, Coverity, and Checkmarx provide assistance, but static analysis frequently generates false positives

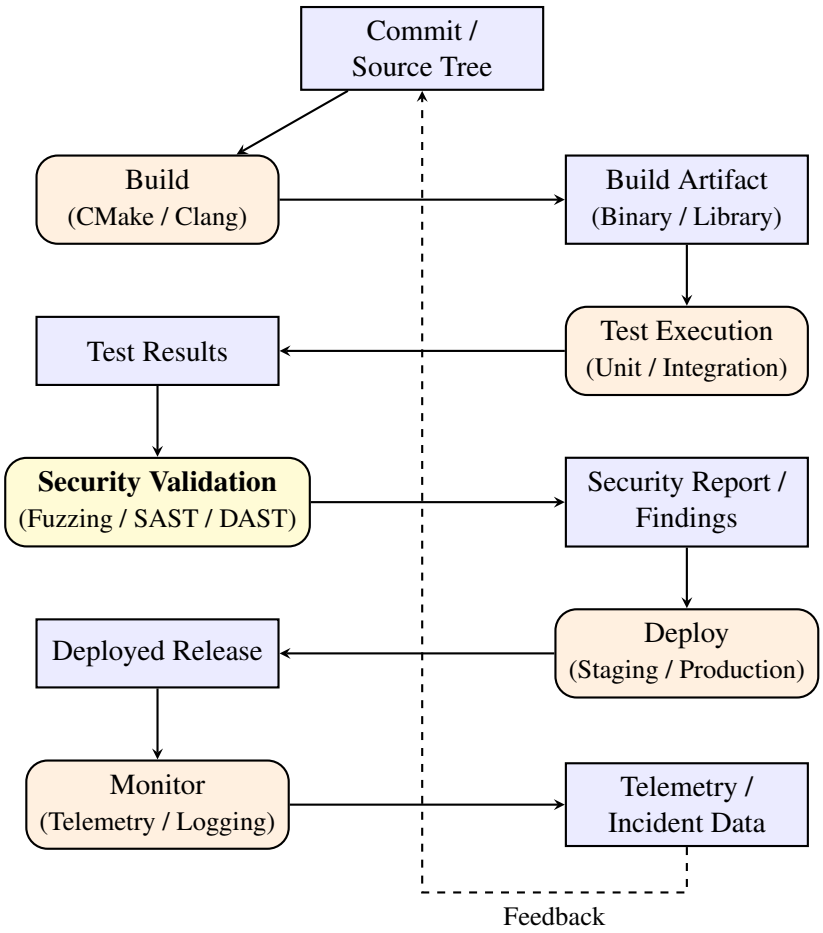

Figure 1.2: The Automotive CI/CD/CT Pipeline. Data artifacts are shown in rectangular boxes; processing steps are shown in rounded boxes. Security validation (highlighted) is where fuzzing integrates into the pipeline. The feedback loop from telemetry to the source tree represents the continuous improvement cycle.

that require manual human triage, and dynamic analysis requires test harnesses that must be created manually.

Fuzzing attempts to bypass this manual review bottleneck, though it introduces a different limitation. Google's OSS-Fuzz project has found over 10,000 vulnerabilities in open-source software through continuous fuzzing [12, 37]. The effectiveness of this approach is extensively documented. The constraint is fuzz driver creation.

A fuzz driver is a small program that feeds generated inputs to target functionality. Writing effective drivers requires understanding API contracts, identifying interesting code paths, and constructing inputs that explore edge cases. This is specialized work. While the automotive software industry employs specialists capable of writing these drivers, the available workforce cannot scale to match the massive volume of daily code production required by major manufacturers.

The mismatch creates a security gap. Code ships with inadequate fuzzing coverage. Vulnerabilities that could have been discovered through systematic fuzzing remain hidden until attackers find them or security incidents force discovery.

Large Language Models offer a potential solution. These models, trained on vast amounts of source code, can generate syntactically correct code following common patterns [38, 4]. Recent research shows LLM capabilities in various software engineering tasks: code completion, bug detection, test generation, documentation [11, 42].

The question we asked was simple: can LLMs generate fuzz drivers? Specifically, can they generate drivers that compile, execute without crashing, and achieve meaningful code coverage?

This question mattered to us at CARIAD because if the answer was yes, it could change how we approach security testing at scale.

1.3 Research Questions and Objectives

We had three main research questions driving this work.

1.3.1 Primary Research Questions

RQ1 (Effectiveness): Can Large Language Models generate fuzz drivers for C++ code that compile successfully, execute without errors, and achieve meaningful code coverage?

This is the fundamental question. A driver that fails to compile is useless. One that compiles but crashes immediately provides no security value. A driver that runs but only exercises trivial code paths does not justify its existence. Meaningful coverage means reaching error handling paths, boundary conditions, and complex interaction sequences [45].

RQ2 (Optimization): Does model size determine fuzz driver quality, or can smaller models with domain-specific training match or exceed larger general purpose models?

Everyone in machine learning hears that bigger models are better. Models with more parameters consistently outperform smaller ones on standard benchmarks [24]. The largest models show impressive capabilities across diverse tasks. They are also expensive (cloud API costs of $0.01–$0.06 per 1k tokens, or dedicated GPU hardware costing $10,000+ per unit), and often have rate limits. But domain-specific tuning offers an alternative path [20]. Instead of relying on the general knowledge of massive models, we adapt smaller models to specific tasks.

We wanted to know: which approach wins for fuzz driver generation?

RQ3 (Feasibility): Can LLM-assisted fuzz driver generation be integrated into secure enterprise CI/CD/CT pipelines that are isolated from external networks while meeting performance, cost, and security requirements?

Academic research often evaluates techniques in isolation, abstracting away deployment concerns. Production matters. Can the solution run within CI/CD/CT time constraints? Does the cost fit budget limits? Can it operate under network security policies that restrict external communication? Is operational complexity acceptable?

These questions drove our entire investigation. We were not just asking whether LLMs could generate code. We were asking whether this approach could work in a real company, dealing with real constraints.

1.4 Thesis Organization

The rest of this thesis is structured as follows.

Chapter 2: Literature Review surveys relevant research. It examines traditional AI approaches in software testing, traces the evolution of Large Language Models and their application to code generation, reviews fuzzing techniques from foundational methods through AI-enhanced approaches, and examines prior work on security integration in CI/CD/CT pipelines.

Chapter 3: Methodology and Framework presents our approach. It introduces the conceptual framework for AI-driven fuzzing using white-box techniques, describes our technical architecture design, and outlines the three research phases: local LLM evaluation, model optimization with LoRA, and enterprise CI/CD/CT integration. We specify evaluation methodology and metrics.

Chapter 4: Implementation covers the concrete realization. It addresses toolchain selection, describes experimental setup for each phase, documents architectural challenges encountered during enterprise integration, and explains how we solved them.

Chapter 5: Experimental Results presents empirical findings: results of fuzz driver generation across evaluated models, performance variations across target repositories, model optimization outcomes, and economic analysis.

Chapter 6: Discussion and Conclusion interprets results against our research questions. It examines what the data reveals about LLM capabilities, addresses unexpected findings, acknowledges limitations, explores implications for automotive industry practice, synthesizes contributions, and identifies future research directions.

2 Literature Review

2.1 Traditional AI Approaches in Software Testing

Automated software testing has been a research area for decades, and AI techniques have played various roles in its evolution. Understanding the historical context helps explain why LLM-based approaches represent a different class of methods.

Genetic algorithms (a class of evolutionary computation, not to be confused with generic algorithms) were among the earliest AI applications to testing. The basic idea was straightforward: treat test case generation as an optimization problem. Maintain a population of test cases, apply selection and mutation operations across generations, and let evolution guide toward more effective tests. Wang et al. documented how evolutionary testing dominated research from the late 1990s through the early 2010s [41].

This approach showed initial promise. Coverage could be formulated as a fitness function. Generate test cases that maximize coverage. Multi-objective optimization helped balance competing goals. Over time, limitations became apparent. Fitness function design proved challenging. Maximizing code coverage did not necessarily correlate with bug detection. Most fundamentally, genetic algorithms lacked semantic understanding. They could shuffle bytes and hope for improvements, but they could not understand what made certain inputs meaningful for a particular API.

Symbolic execution offered a different approach based on constraint solving rather than random search. The technique treats program inputs as symbolic values and accumulates path constraints during execution. Solving these constraints yields concrete inputs guaranteed to exercise specific program paths. Microsoft's SAGE, deployed internally for security testing, demonstrated practical viability by finding vulnerabilities that conventional testing had missed [15].

Symbolic execution offers theoretical rigor. But it faces serious scalability problems. The number of paths through a program grows exponentially with the number of branch points. This is the path explosion problem. Yavuz's recent work on DESTINA addresses this through targeted execution strategies, but complete path coverage remains infeasible for realistic programs [44].

Machine learning entered testing in supporting roles around 2015. A Sandia National Laboratories report surveyed early applications: defect prediction, test case prioritization, coverage estimation [36]. These applications used ML models to inform human decisions rather than generate tests directly. They were practical and

useful but did not fundamentally change how tests were created.

Neural networks enabled more ambitious applications. Pei et al. introduced DeepXplore for testing deep learning systems, defining neuron coverage as a testing adequacy criterion [35]. Odena et al. extended coverage-guided fuzzing to neural networks with TensorFuzz [32]. These works showed that neural networks could participate actively in testing rather than merely supporting it.

This context shows why LLM-based approaches represent a different class of methods. Previous AI techniques either optimized based on metrics (genetic algorithms, coverage-guided fuzzing) or reasoned about program structure (symbolic execution). LLMs do something different: they learn patterns from vast code repositories and can generate syntactically correct code following those patterns.

2.2 Large Language Models: Evolution and Code Generation

The Transformer architecture, introduced in 2017, enabled training on vastly larger datasets than previous approaches permitted. Scaling revealed that models trained on next-token prediction could perform diverse tasks.

GPT-3 (2020) demonstrated this principle. With 175 billion parameters, it could write coherent text and syntactically valid code. The specific finding that mattered for software engineering: the model had learned patterns from code in its training data and could apply them to generate new code.

Code-specific models followed. Codex, which powers GitHub Copilot, achieved 28.8% success on the HumanEval benchmark initially and 70.2% with multiple sampling attempts [8]. GPT-4 further improved code generation with better contextual understanding and reduced subtle errors. Context windows expanded from 2K to 128K tokens, enabling inclusion of multiple source files, headers, and documentation in prompts.

Open-source alternatives emerged as major counterparts. Meta's LLaMA showed that competitive performance was achievable with smaller, publicly available models [38]. Code-specific variants like StarCoder and CodeLlama targeted programming tasks specifically. This mattered for automotive applications where intellectual property constraints make sending code to external services problematic.

Researchers have systematically tested LLM code generation. Deng et al. found that models generated syntactically valid code at high rates but frequently contained semantic issues: code that compiled but exercised APIs incorrectly [11]. Wang et al. observed similar patterns in mutation testing experiments, noting high success on simple cases with degrading performance as complexity increased [40].

C presents particular challenges for LLM-based code generation. Training data skews heavily toward Python, JavaScript, and other high-level languages. C's distinc-

tive features, such as pointer arithmetic, manual memory management, preprocessor macros, and undefined behavior, are less well represented in training data. Empirical evaluations consistently show worse performance on C compared to Python [45]. This has direct implications for automotive applications where C remains the dominant language for embedded systems.

2.2.1 Low-Rank Adaptation (LoRA)

Fine-tuning a pre-trained LLM for a downstream task traditionally requires updating all model parameters, which demands the same memory and compute resources as the original training. Hu et al. proposed *Low-Rank Adaptation* (LoRA) as a parameter-efficient alternative [20].

The key insight behind LoRA is that the weight updates during fine-tuning exhibit low intrinsic dimensionality. Instead of updating a full weight matrix $W \in \mathbb{R}^{d\times k}$, LoRA freezes W and injects a trainable low-rank decomposition $\Delta W = BA$, where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with rank $r \ll \min(d,k)$. During the forward pass the modified output becomes:

$$h = Wx + \Delta Wx = Wx + BAx$$

Only A and B are trained; the original weights W remain frozen. This reduces the number of trainable parameters by several orders of magnitude. For example, with $d = k = 4096$ and $r = 16$, full fine-tuning updates $\approx$16.8 M parameters per layer, whereas LoRA updates only $2 \times 4096 \times 16 = 131$ K, a $128\times$ reduction.

LoRA is typically applied to the query and value projection matrices (W_q, W_v) of the Transformer attention layers, though it can be extended to all projection matrices (W_k, W_o) for greater expressiveness [20]. A scaling factor α/r is applied to ΔW to control the magnitude of the adaptation relative to the pre-trained weights.

This approach has two practical advantages for our work. First, it enables fine-tuning on consumer-grade hardware: a 1.5B-parameter model can be adapted in hours on a single GPU with 16 GB VRAM. Second, the LoRA adapters can be saved as lightweight files ($\sim$30 MB) and loaded on top of the frozen base model at inference time, avoiding the need to distribute a separate full model checkpoint.

2.3 Foundations of AI-Guided Fuzzing Techniques

Fuzzing originated with Barton Miller's 1988 experiments at the University of Wisconsin, where students subjected UNIX utilities to streams of random input and found that approximately 25% crashed or hung [30]. This established fuzzing as a viable technique for uncovering reliability issues. Early fuzzing was entirely random, requiring no knowledge of target program structure. But simplicity had a cost: random inputs rarely satisfy validation checks and prevent exploration of deeper logic.

Coverage-guided fuzzing represented a fundamental advance. AFL (American Fuzzy Lop), released in 2013, introduced lightweight instrumentation to track which code paths each input explored. Inputs discovering new coverage were retained and mutated further. Inputs revealing no new coverage were discarded. This feedback loop dramatically improved efficiency by directing search toward unexplored behavior [1].

AFL spawned an ecosystem. AFL++ added performance optimizations and additional mutation strategies. LibFuzzer integrated coverage-guided fuzzing into the LLVM toolchain [37]. Syzkaller adapted coverage-guided techniques for kernel testing [17]. Google's OSS-Fuzz project deployed these tools at scale, continuously fuzzing hundreds of open-source projects.

Ding and Le Goues analyzed bugs found by OSS-Fuzz, documenting vulnerability classes that other testing methods had missed [12]. The technique works. The bottleneck is harness creation.

Grammar-based fuzzing addressed structural validity problems. Random byte mutations rarely produce syntactically valid inputs for complex formats. A randomly mutated JSON file is almost certainly invalid JSON. Grammar-based fuzzers generate inputs according to specified grammars, ensuring structural validity while exploring variations. This proved particularly effective for protocol parsers and file format handlers.

Despite these advances, challenges persist in fuzzing practice. Writing fuzz harnesses connecting fuzzers to target libraries remains manual and labor-intensive. Coverage plateaus occur when random mutation cannot discover paths beyond certain checkpoints. These limitations motivate AI-enhanced approaches that can reason about program structure.

2.4 AI-Enhanced Fuzzing: State of the Art

Combining machine learning and fuzzing has led to a growing body of research. This section surveys the current state.

LLM-Based Test Case Generation represents a fundamentally different approach. Rather than mutating existing inputs, LLMs generate tests from specifications, documentation, or source code.

Deng et al. introduced TitanFuzz, demonstrating that LLMs could generate effective fuzz targets for deep learning library APIs by prompting models with API documentation [11]. TitanFuzz produced test cases achieving higher coverage than manually written tests for several libraries and discovered previously unknown bugs. This demonstrated that tests produced by LLMs can be applied in real projects.

FuzzGPT extended this approach with focus on edge cases, observing that LLMs possess implicit knowledge of typical API usage patterns from training on vast code repositories [10]. By prompting specifically for atypical or boundary case inputs,

FuzzGPT discovered bugs that conventional fuzzers overlooked.

Fuzz4All pursued a more general approach, targeting multiple programming languages and application domains while using LLMs to generate both test inputs and harness code [42].

Zhang et al. directly evaluated LLM-generated fuzz drivers compared to manually written alternatives, finding that results varied significantly across libraries [45]. Some LLM drivers achieved 80 to 90 percent of manual driver coverage. Others reached only 50%. Performance correlated with documentation quality and API complexity.

Previous work worth noting includes HyLLfuzz, which combines LLM-generated seeds with mutation-based fuzzing [27], and WhiteFox, which uses LLMs for targeted compiler fuzzing [43]. These are hybrid approaches attempting to combine LLM strengths while mitigating limitations.

Across these studies, authors repeatedly note a gap between code that compiles and code that does something meaningful. LLMs produce code that compiles at high rates but may not exercise meaningful behavior.

2.4.1 Research Gap of LLM-Based Fuzzing Techniques

While the systems surveyed above demonstrate that LLMs can generate fuzz targets and even fuzz drivers in controlled settings (e.g., TitanFuzz, FuzzGPT, and Fuzz4All) [11, 10, 42], they do not constitute a drop-in solution for the setting of this thesis: automated generation of C/C++ libFuzzer-style drivers under enterprise and automotive constraints. In particular, prior work repeatedly highlights that compilation success is not equivalent to meaningful test behavior [45]. For fuzzing, this is a critical limitation: a driver can compile yet still abort immediately due to invalid API usage, execute only shallow "happy-path" code, or miss error-handling and boundary-condition paths that are most relevant for security.

Beyond this "compiles vs. meaningful coverage" gap, several practical constraints prevent directly adopting existing techniques in our environment. Many approaches implicitly assume high-quality, complete API documentation to prompt the model, even though real-world C++ libraries often expose large, template-heavy surfaces with incomplete documentation. Others rely on cloud-based model access, which conflicts with IP protection requirements and network-isolated CI runners. Still others follow a research-style workflow where manual prompt iteration and human correction are acceptable, rather than a repeatable CI/CD/CT-compatible process with predictable latency and cost. Hybrid approaches such as HyLLfuzz show the value of combining LLM generation with mutation-based fuzzing [27], but they still leave open how to systematically operationalize the approach across targets, models, and enterprise deployment constraints.

This thesis therefore proposes an end-to-end technique that retains the core idea of LLM-based harness/driver generation from API context [11, 42] and the hybrid prin-

ciple of letting conventional coverage-guided fuzzing do the large-scale exploration [27], while adding the missing components needed in practice: automated context extraction, automated compilation and sanitizer validation, and coverage-based evaluation gates to ensure generated drivers are not merely syntactically valid but also behaviorally useful. Chapter 3 presents this method in detail.

2.5 CI/CD/CT Pipeline Security Integration

Continuous Integration (CI), Continuous Delivery/Deployment (CD), and Continuous Testing (CT) have become foundational practices for managing complexity while maintaining development velocity. The philosophy is straightforward: integrate code changes frequently, test automatically, deploy validated changes rapidly.

The automotive industry has adapted these principles to domain-specific constraints. Formal release processes with human review precede deployment to production vehicles. Nevertheless, core principles apply: automate what can be automated, test early and often, maintain a continuously releasable codebase [25].

This creates tension with thorough security testing. A well-optimized pipeline might complete build and basic testing in 15 to 30 minutes. Developers expect rapid feedback. Security testing, particularly fuzzing, requires extended execution to achieve meaningful coverage. A fuzzing campaign might run for hours or days. This timescale is incompatible with developer expectations [25].

Enterprise environments add infrastructure complexity. Large organizations deploy systems for CI/CD/CT across hybrid cloud architectures, balancing computational demands against security requirements. Build runners may execute in public cloud environments for scalability while sensitive operations stay in private networks isolated from external connections. Network policies restrict communication between zones.

We should note that when we started this work, we underestimated how much this infrastructure complexity would matter. We thought the hard problem was generating good fuzz drivers with LLMs. The primary challenge lies in deploying these models within secure enterprise networks, where strict data privacy requirements and isolated infrastructure often restrict the theoretical benefits of Large Language Models.

2.6 Automotive Software Development and Safety Standards

Automotive software operates under regulatory constraints unlike most other domains. Safety standards like ISO 26262 require systematic identification and mitigation of safety risks throughout the vehicle lifecycle [22]. Every failure mode must

be analyzed. Fixes must be verified through testing. This fits safety engineering, where hazards are defined and the physical behavior is predictable.

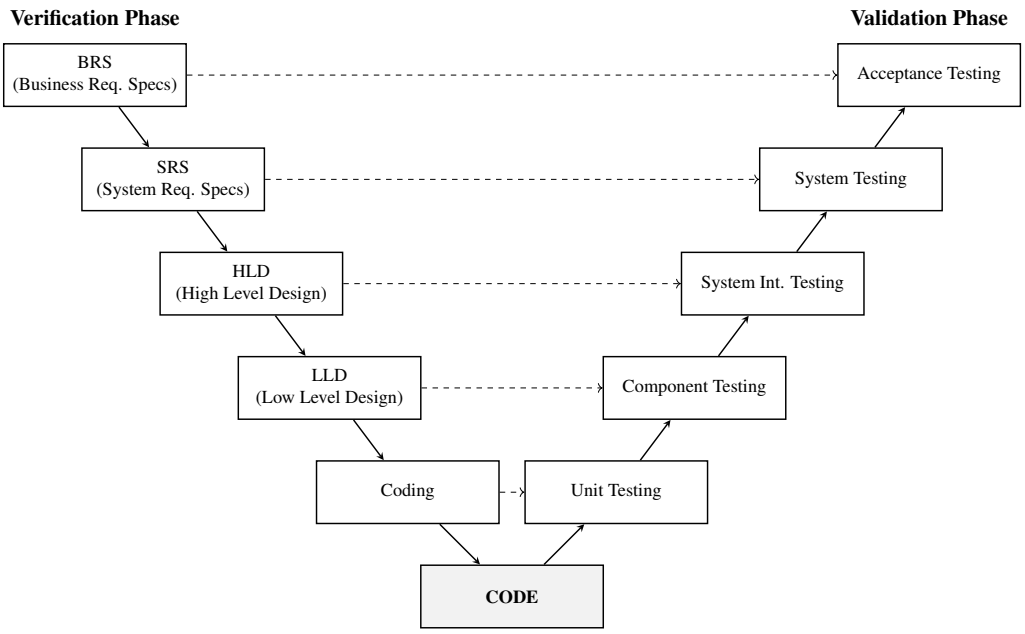

Figure 2.1: The V-Model Development Lifecycle (adapted from ISO 26262 [22]). The diagram illustrates the relationship between each development phase (left) and its corresponding testing phase (right), converging at the Code implementation.

Security engineering faces a different problem. Attackers are intelligent, creative, and unconstrained by specifications. UNECE Regulation 155 and ISO/SAE 21434 require cybersecurity management systems and risk assessment, but implementation requires different approaches than safety engineering [23]. The expected behavior must be verified, but also the unexpected behavior must be tested for weaknesses.

As illustrated in the V-Model (Figure 2.1), the standard lifecycle assumes design and testing are perfectly matched. But security breaks this assumption. Safety requires comprehensive coverage of specified behaviors; security requires exploration of unspecified attack surfaces. Both demand rigorous testing. Development cycles must accelerate while testing becomes more thorough. Under these conditions, AI-assisted fuzzing is no longer a theoretical idea but a practical need.

2.7 Research Gaps and Thesis Positioning

Based on this review, several specific gaps can be identified.

First, while previous work has evaluated LLM-based test generation on individual target libraries, comprehensive evaluation across diverse targets with different models

and with explicit analysis of model scale versus specialization is limited. We address this through systematic evaluation of 14 models across multiple C++ libraries.

Second, while optimization techniques like LoRA are well-established in general LLM domains, their application to fuzzing is underexplored [20]. We investigate whether domain-specific fine-tuning of small models can match or exceed larger general purpose models.

Third, academic research typically evaluates techniques in isolation. Production deployment in secure enterprise environments faces systemic engineering challenges, such as integrating non deterministic models into automated continuous integration pipelines, managing data privacy constraints without compromising model context, and addressing latency within closed networks. We focus on bridging this gap between theoretical evaluation and practical engineering deployment.

Finally, while prior work examines success cases, systematic analysis of where generated drivers fail and detection mechanisms is limited.

This thesis addresses these gaps through systematic evaluation and explicit attention to enterprise deployment challenges.

3 Methodology and Framework

In this chapter, we explain how the research was carried out, from early experiments to the final deployment at CARIAD SE between May and September 2025. We explain the conceptual foundations of our framework, the technical design that supports it, and the three-phase research strategy that guided our work from initial experiments to enterprise deployment.

The methodology was primarily shaped by the need to rigorously evaluate the application of large language models within complex software security contexts. The research design had to adapt to the rapid advancements in generative AI, where new models and capabilities emerge continuously. Most importantly, the methodology aimed to produce generalizable empirical results that contribute to the broader scientific understanding of automated vulnerability discovery and guide future research directions.

3.1 Conceptual Framework: AI-Driven White-Box Fuzzing

The conceptual framework for this research combines established ideas from software security testing with new possibilities created by large language models. To explain our approach clearly, we first review the core problem and our proposed solution.

3.1.1 The Fuzz Driver Problem

Fuzzing is an effective technique for finding vulnerabilities, but it relies on a critical component: the fuzz driver. A fuzzer like libFuzzer generates raw inputs (byte streams). A fuzz driver is a C++ program that translates those inputs into meaningful API calls to the target library. It acts as the bridge between the random mutations of the fuzzer and the logic of the library.

Writing effective drivers requires deep understanding of the API. An engineer must know what functions exist, what parameters they accept, what preconditions must be satisfied, and what edge cases might cause problems. This understanding represents specialized domain knowledge. Creating effective fuzz drivers typically requires 2 to 8 hours of specialized engineering work per target library (lower end for well-

documented parsing APIs, upper end for complex C++ libraries with heavy template usage), making it a resource-intensive process that is difficult to scale manually against the high volume of modern software production.

3.1.2 Our Approach: Automated Generation

We investigated whether Large Language Models (LLMs) could automate this process. The conceptual idea is straightforward: feed an LLM a description of a target library API (header files, documentation) and ask it to generate a valid fuzz driver.

If the model produces compilable, executable code that achieves meaningful coverage, we have partially automated what previously required human effort. This addresses the bottleneck directly. Instead of assigning a specialist to write each driver manually, we run an automated pipeline, review the output, and deploy it.

We know LLMs have constraints. LLMs sometimes generate code that compiles but does not exercise interesting behavior. They sometimes misunderstand API contracts. They sometimes generate code with undefined behavior that crashes the fuzzer immediately. These are not theoretical concerns; we encountered all of them during our work.

We recognized that imperfect automation still helps. If an LLM generates a driver that achieves 40% coverage and saves 2 to 8 hours of expert time, that is progress. We do not need perfection. We define a generated driver as *sufficient* if it meets three criteria: it compiles without errors, it executes without crashes or sanitizer violations, and it achieves at least 30% line coverage of the target library's public API surface. A driver meeting these criteria provides an automated baseline that can be refined further, replacing manual effort that would otherwise cost 2 to 8 hours per target.

3.2 Technical Architecture Design

Moving from concept to working system required a robust technical design. Our system uses a layered architecture with three main layers: the **Input Layer**, the **Generation Layer**, and the **Execution Layer** (corresponding to Sections 3.2.1–3.2.3). Each layer is marked in Figure 3.1.

As shown in Figure 3.1, the source code and its public header files serve as the primary input to the pipeline. These inputs are processed through three successive layers (context extraction, LLM-based generation, and automated execution) to ultimately produce coverage reports and crash data, replacing the manual fuzz driver creation step shown in Figure 1.1.

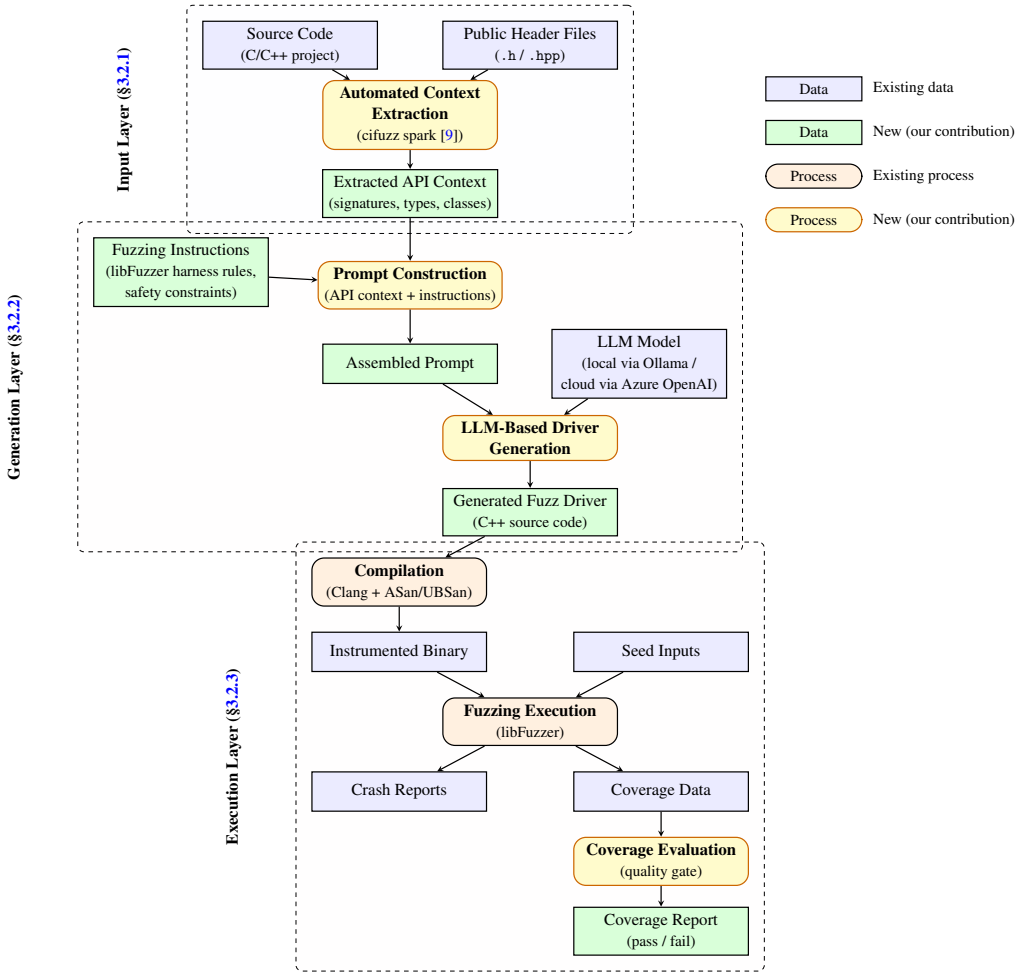

Figure 3.1: Technical Architecture of our LLM-based fuzz driver generation pipeline. The three layers are marked with dashed borders. Data artifacts are shown in rectangular boxes; processing steps in boxes with rounded corners. Green/yellow highlighted elements denote new contributions of this work with respect to the state of the art (cf. Figure 1.1).

3.2.1 Input Layer (Automated Context Extraction)

The first layer prepares the context that the LLM needs to generate a meaningful fuzz driver. Instead of writing custom parsers, we used the analysis capabilities of **cifuzz spark**, a module of the cifuzz fuzzing toolkit developed by Code Intelligence [9]. Given a CMake-based C/C++ project, cifuzz spark automatically parses the project structure, identifying public headers and API surfaces relevant for testing. It traverses the abstract syntax tree (AST) to extract function signatures, class definitions, type aliases, and parameter types. The output is the *extracted API context*: a structured summary of the target library's public interface, including function names, their parameter types, return types, and enclosing class or namespace information. This ensures the LLM receives accurate and complete context about the code it needs to test, without requiring manual intervention or maintenance of custom extraction scripts.

3.2.2 Generation Layer (LLM Interface)

This layer handles the interaction with the LLM and the construction of the prompt that drives driver generation. It includes two critical sub-systems:

- **Prompt Construction:** We combine the *extracted API context* (the structured function signatures, types, and class definitions produced by the Input Layer) with a set of *fuzzing instructions*. These fuzzing instructions are predefined rules and constraints that describe what a valid libFuzzer fuzz driver must look like. Specifically, they define the required *fuzz test case structure*: the driver must implement the `LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)` entry point, correctly consume the fuzzer-provided byte buffer, translate those bytes into valid API calls, and avoid trivial or undefined behavior. The instructions also encode safety constraints (e.g., do not call `exit()`, avoid infinite loops) to ensure the generated driver is compatible with the fuzzing engine.

- **Backend Abstraction:** We configured the *environment*, that is, the runtime configuration that determines which LLM backend serves the generation requests, to swap between different model backends. While cifuzz manages the overall workflow, we routed the generation requests to both local models (via Ollama) and cloud-hosted models (via Azure OpenAI) to compare performance across different architectures. This abstraction allows the same pipeline to operate in network-isolated environments (using local models) or leverage more powerful cloud models when network access is available.

3.2.3 Execution Layer (Evaluation Pipeline)

The generated fuzz drivers, that is, the C++ source files produced by the LLM in the Generation Layer, are useless if they do not compile and run correctly. This layer

handles the automated compilation, execution, and evaluation pipeline:

1. **Compilation and Validation:** The generated driver source code is compiled against the target library using Clang. This step verifies that the LLM-produced code is syntactically valid and links correctly against the target API.

2. **Sanitizer Instrumentation:** The driver is recompiled with AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) enabled. This detects memory errors, buffer overflows, and undefined behavior that might not cause an immediate crash but indicate incorrect API usage.

3. **Fuzzing Execution:** If compilation succeeds, the instrumented binary runs under `libFuzzer` together with any available seed inputs. The fuzzing engine monitors coverage metrics and records any crashes.

4. **Coverage Evaluation:** After the fuzzing run completes, the collected coverage data is evaluated against a quality gate to determine whether the generated driver achieves meaningful coverage of the target library, ensuring the driver is not merely syntactically valid but also behaviorally useful.

3.3 Research Strategy: Three-Phase Approach

Our research unfolded in three distinct phases, each addressing different aspects of the research questions defined in Chapter 1.

3.3.1 Phase 1: Local LLM Evaluation

We gathered 14 open-source LLM models ranging from 1.5B to 46.7B parameters and evaluated them on multiple C++ target libraries. This phase directly addresses **RQ1** and **RQ2**. Can LLMs generate useful fuzz drivers? Does model size matter? Which models work best?

We chose open-source models specifically because automotive applications have intellectual property constraints. Sending proprietary code to external APIs is often problematic. Local models, run on available hardware, avoid this concern entirely.

3.3.2 Phase 2: Model Optimization with LoRA

Based on Phase 1 results, we selected a promising small model and applied Low Rank Adaptation (LoRA) fine-tuning. We curated a dataset of high-quality fuzz drivers from the Google OSS-Fuzz project to teach the model the specific structure of a good test harness.

The goal was to adapt the model specifically for the fuzzing task while reducing computational cost. This phase addresses **RQ2** more directly: can smaller, fine-tuned models outperform larger general purpose models?

3.3.3 Phase 3: Enterprise CI/CD/CT Integration

The final phase moved from controlled experiments to the real world. We integrated the solution into CARIAD's actual CI/CD/CT infrastructure. This phase addresses **RQ3**: Can this approach work in a real corporate environment with real network security policies, cost constraints, and operational complexity?

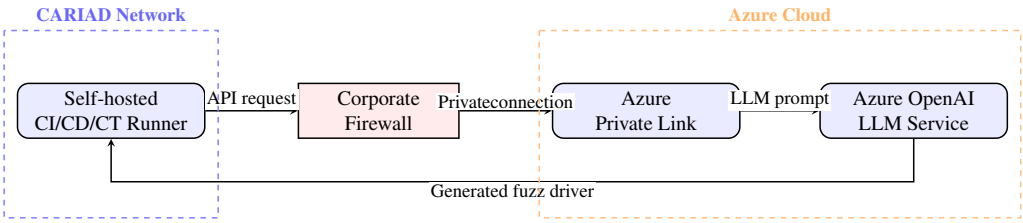

Figure 3.2: Enterprise Integration Strategy. The self-hosted CI/CD/CT runner communicates with Azure OpenAI through a Private Link endpoint, routing traffic through the corporate firewall over a private connection without traversing the public internet.

As illustrated in Figure 3.2, this phase revealed challenges we had not anticipated during earlier work. The corporate firewall blocked external API calls, requiring us to re-architect the solution using Azure Private Link.

3.4 Evaluation Methodology

We evaluated generated fuzz drivers using quantitative metrics to ensure objectivity, focusing on both code quality and pipeline impact.

- **Compilation Success:** Does the driver compile against the target library? This is the first and most critical filter.
- **Runtime Behavior:** Does the driver execute without crashing immediately? We checked if the driver actually runs or just fails instantly.
- **Code Coverage:** What percentage of the target library does the driver test? We measured this using standard tools (`llvm-cov`).
- **Vulnerability Discovery (MTTV):** We measured the Mean Time To Vulnerability, that is, how long the fuzzer needs to run before it finds a bug.

- **Pipeline Latency (CI Overhead):** We measured how much extra time the AI adds to the build process. The system must stay within CARIAD's CI/CD/CT feedback window (see Chapter 4, Section 4.4 for the specific performance requirements).
- **Cost and Efficiency:** We calculated the exact price of generating a driver. By tracking the number of tokens (words) sent to the AI, we determined the cost in euros to prove it is cheaper than human engineering.

For each target library, we also manually inspected a sample of generated drivers to understand failure modes. We documented cases where drivers compiled but did not work as intended. Finally, we conducted a cost analysis for enterprise deployment, modeling different scenarios to estimate annual expenses.

4 Implementation

We turned the methodology from Chapter 3 into working code and systems over the course of this implementation. The three research phases defined in Chapter 3 (Section 3.3), namely Phase 1: Local LLM Evaluation (Section 4.2), Phase 2: Model Optimization with LoRA (Section 4.3), and Phase 3: Enterprise CI/CD/CT Integration (Section 4.4), map directly to the development progression described below. Each phase built on the previous one. The following sections cover our tool choices, the specific steps we followed, and the engineering decisions we made along the way.

4.1 Toolchain Selection and Rationale

Before writing any code, we needed to select the appropriate tools. Several factors guided the selection. Research reproducibility required tools with stable APIs and clear documentation. Industrial deployment required compatibility with CARIAD's existing infrastructure. Hardware specifications influenced which models could run locally with acceptable performance.

4.1.1 Fuzzing Infrastructure

For the fuzzing engine itself, we chose libFuzzer combined with Clang sanitizers. libFuzzer is the industry standard for coverage-guided fuzzing in C and C++ projects [37], though alternatives like AFL++ [1] also have strong communities. libFuzzer integrates directly with the LLVM toolchain, which means compilation and fuzzing use the same infrastructure. Google uses libFuzzer for OSS-Fuzz [12], so the documentation and community support are excellent.

We compiled all targets with AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) enabled. ASan catches memory errors like buffer overflows, use-after-free, and memory leaks [37]. UBSan catches undefined behavior like signed integer overflow and null pointer dereferences. These sanitizers add runtime overhead, but for security testing the tradeoff is acceptable, since detecting vulnerabilities takes priority over execution speed.

For integration with our build systems, we used cifuzz from Code Intelligence. The tool wraps libFuzzer and provides additional functionality like corpus management, coverage reporting, and build system integration. The most relevant feature for our work was cifuzz spark, which handles the LLM integration. Spark takes a CMake

target as input, extracts API information automatically, sends it to the configured LLM, and produces a fuzz driver ready for execution.

Our choice of cifuzz was based on a systematic evaluation. We compared several alternatives including Google's ClusterFuzz [12] and standalone libFuzzer setups. Among these, cifuzz offered a good balance between ease of use and enterprise features. Its spark module specifically addressed our core research question by providing a clean interface between the fuzzing infrastructure and LLM backends.

4.1.2 LLM Infrastructure

We needed to run models locally for several reasons. First, evaluating many different models through paid APIs would have been cost-prohibitive. Second, some models are open-source and only available for local deployment. Third, understanding local deployment options was essential for the enterprise integration phase.

We selected Ollama [33] as our primary local inference server. Ollama simplifies model management. Installing a new model is one command: `ollama pull qwen2.5-coder:32b`. The server exposes an OpenAI-compatible API, which meant cifuzz spark worked without modification. Ollama handled quantized models well. Quantization compresses models to use reduced numerical precision (e.g., 4-bit instead of 16-bit weights), which let us run larger models on the available hardware.

For models not available through Ollama, we used llama.cpp directly. This required more manual configuration but provided additional flexibility. Some models from Hugging Face came as split GGUF files that needed merging before use. We handled this with the following command:

```
llama-gguf-split --merge qwen2.5-coder-14b-instruct-q5_k_m
    -00001-of-00002.gguf \
                    qwen2.5-coder-14b-instruct-q5_k_m.gguf
```

Listing 4.1: Merging Split GGUF Model Files

Merging these split files added an extra step to our model preparation process.

For cloud deployment in the enterprise phase, we configured Azure OpenAI [28]. Azure provided GPT-4o through a managed endpoint. The authentication used API keys stored as GitHub secrets. We configured the following environment variables:

```
export CIFUZZ_LLM_API_URL="https://<endpoint>.openai.azure.com"
export CIFUZZ_LLM_AZURE_DEPLOYMENT_NAME="gpt-4o"
export CIFUZZ_LLM_API_TOKEN="<api-key>"
export CIFUZZ_LLM_MAX_TOKENS=40000
```

Listing 4.2: Azure OpenAI Configuration

A token limit of 40,000 was necessary because fuzz driver generation requires extensive context. A typical prompt includes the extracted API context (header files with function signatures), fuzzing instructions (libFuzzer entry-point requirements,

safety constraints), and optionally example driver snippets. Smaller token limits led to truncation and reduced output quality. A representative example prompt is provided in Appendix A.5.

4.1.3 Development Environment

Local development happened on a MacBook Pro with an M1 Pro chip. For Linux container support on Apple Silicon, we chose Podman as our container runtime. Podman aligned well with the enterprise CI/CD/CT runners, which also used Linux.

```
brew install podman
podman machine init --cpus 4 --memory 8192
podman machine start
```

Listing 4.3: Podman Setup on macOS

Transitioning from Docker to Podman involved adapting several scripts to align with Podman's container networking model.

For the enterprise environment, we used CARIAD's internal GitHub Enterprise instance with self-hosted runners. These runners used Buildah [6] instead of Docker for container operations. Buildah is daemonless, which complies with security policies that govern long-running daemon processes.

Our build toolchain used CMake for project configuration and Clang for compilation. We required CMake version 3.5 or higher because cifuzz integration depends on features from that version. The CMake configuration for enabling fuzzing required minimal setup:

```
cmake_minimum_required(VERSION 3.5)
cmake_policy(VERSION 3.5)

project(FuzzTarget)

find_package(cifuzz NO_SYSTEM_ENVIRONMENT_PATH)
enable_fuzz_testing()
add_subdirectory(cifuzz-spark)
```

Listing 4.4: CMake Fuzzing Configuration

We also needed to update CMakeLists.txt files in subdirectories. Several target libraries used CMake configurations that needed updates for compatibility with our setup. The fix was consistent across all subdirectories:

```
cmake_minimum_required(VERSION 3.10...3.22)
cmake_policy(VERSION 3.10...3.22)
```

Listing 4.5: Subdirectory CMake Updates

The LLM-assisted fuzzing workflow, conceptually introduced in Chapter 3, demonstrates how LLMs integrate into the fuzzing pipeline to automatically generate test

cases from source code. Our implementation uses cifuzz spark to handle the integration between the LLM backends (either local models via Ollama or Azure OpenAI) and the libFuzzer execution environment.

4.2 Phase 1: Local LLM Evaluation Setup

Phase 1 focused on understanding which models could generate useful fuzz drivers. The experiments in this phase target **RQ1** (Can LLMs generate compilable, executable fuzz drivers?) and **RQ2** (Does model size determine quality?). We did not assume any model would work well. Literature suggested mixed results for LLM code generation, and fuzz drivers are a specialized task.

4.2.1 Benchmarked Models

We evaluated 14 models spanning a wide range of sizes and architectures. The selection included both general-purpose models and code-specialized variants. Our goal was to understand whether model size, training data, or architecture had the greatest impact on fuzz driver quality.

Large Models (25 to 35 billion parameters):

- Qwen 2.5-Coder 32B
- DeepSeek Coder 33B
- CodeLlama 34B Instruct
- WizardCoder 34B
- Yi 34B
- Gemma 3 27B

Extra Large Models (above 40 billion parameters):

- Mixtral 46.7B (Mixture-of-Experts architecture, approximately 12.9B active parameters per inference)

Medium Models (7 to 16 billion parameters):

- Qwen 2.5-Coder 7B and 14B
- Phi-4 14B
- DeepSeek Coder V2 Lite 16B
- Gemma 3 9B

Small Models (below 7 billion parameters):

- Qwen 2.5-Coder 1.5B
- Gemma 3 4B

Code-specialized models like Qwen Coder, DeepSeek Coder, and CodeLlama were trained specifically on programming tasks. General-purpose models like Mixtral, Yi, Gemma 3, and Phi-4 were included to investigate whether broad training with strong code exposure could compensate for lack of explicit code specialization. This allowed us to test a key hypothesis, whether a 46B general model outperforms a 7B code-specialized model. Results, discussed in Chapter 5, were surprising: the general-purpose Yi (34B) achieved 0% code coverage at runtime, and Mixtral (46.7B) could not be evaluated at all due to hardware constraints, while Qwen 2.5-Coder (32B) and Gemma 3 (27B) consistently achieved over 43% line coverage. Even some code-specialized models like CodeLlama (34B) did not produce working drivers, suggesting that neither code specialization nor model size alone is sufficient without adequate training on relevant code patterns.

Installing these models required substantial disk space: the larger models exceeded 20 GB each in their quantized forms (see Table 4.1 for per-model sizes). We used 4-bit and 5-bit quantization to fit models into available VRAM while maintaining reasonable output quality. A complete summary of all 14 evaluated models, including their parameter counts, quantization levels, architectures, and results, is provided in Table 4.1 and in the appendix (Table A.1).

Table 4.1: Summary of All Evaluated Models on yaml-cpp

Model	Type	Params	Quant.	Disk Size	yaml-cpp Cov.	Status
Qwen 2.5-Coder 32B	Code-specialized	32B	Q4_K_M	∼20 GB	43.08%	Functional
Gemma 3 27B	General-purpose	27B	Q4_K_M	∼17 GB	45.06%	Functional
Phi-4 14B	General-purpose	14B	Q4_K_M	∼9 GB	34.26%	Functional
Qwen 2.5-Coder 14B	Code-specialized	14B	Q5_K_M	∼10 GB	0%	Compile failure
Qwen 2.5-Coder 7B	Code-specialized	7B	Q5_K_M	∼5 GB	0%	Compile failure
Qwen 2.5-Coder 1.5B	Code-specialized	1.5B	FP16	∼3 GB	0%	Compile failure
DeepSeek Coder 33B	Code-specialized	33B	Q4_K_M	∼20 GB	0%	Compile failure
DeepSeek Coder V2 Lite	Code-specialized (MoE)	16B	Q4_K_M	∼10 GB	0%	Compile failure
CodeLlama 34B Instruct	Code-specialized	34B	Q4_K_M	∼20 GB	0%	Compile failure
WizardCoder 34B	Code-specialized	34B	Q4_K_M	∼20 GB	0%	Compile failure
Yi 34B	General-purpose	34B	Q4_K_M	∼20 GB	0%	Invalid API usage
Gemma 3 9B	General-purpose	9B	Q4_K_M	∼6 GB	0%	Compile failure
Gemma 3 4B	General-purpose	4B	Q4_K_M	∼3 GB	0%	Compile failure
Mixtral 46.7B	General-purpose (MoE)	46.7B	Q4_K_M	∼28 GB	N/A	Hardware limit

4.2.2 Target Repositories

Consistent test targets were needed for fair model comparison. Our selection included open-source C++ libraries with varying complexity:

Primary Target: yaml-cpp

yaml-cpp is a YAML parsing library with approximately 35 source files and over 1000 API candidates that could potentially be fuzzed. We chose it as our primary benchmark because it has enough complexity to challenge the models while being well documented and actively maintained. The library also has existing fuzz targets in OSS-Fuzz, giving us a baseline for comparison.

Secondary Targets:

- **RapidJSON**: A JSON parsing library widely used in performance-critical applications. RapidJSON's CMake configuration needed adjustments for our setup, which we resolved by re-downloading a clean version of the source.
- **pugixml**: XML processing library with similar parsing challenges to yaml-cpp.
- **fmt**: Text formatting library with different API patterns than the parsers.
- **spdlog**: Logging library with many configuration options.
- **jsoncons**: Another JSON library for comparison with RapidJSON.

These libraries cover different domains but share the characteristic of being C++ with clean public APIs. They all have existing fuzz targets in OSS-Fuzz, which provided baselines for comparison.

We also tested on several CARIAD internal libraries, though we cannot report those results in detail due to confidentiality constraints. The patterns we observed on open-source libraries were consistent with the results on internal ones.

4.2.3 Evaluation Environment

Hardware for local evaluation was a workstation with sufficient GPU memory for running 32B parameter models with 4-bit quantization. Larger models like Mixtral 46.7B required offloading to system RAM, which increased inference time.

Each model's evaluation followed these steps:

1. Start the model server (Ollama or llama.cpp)
2. Configure cifuzz spark environment variables
3. Run `cifuzz spark` on the target library
4. Attempt to compile the generated driver
5. If compilation succeeded, run the fuzzer for a fixed duration (60 seconds)
6. Record coverage metrics using llvm-cov

We ran each model five times on yaml-cpp to account for stochastic output variation. Preliminary tests showed that variance in coverage results stabilized after three to four runs, so we chose five runs to provide an additional margin of confidence. Some models produced different outputs on identical prompts, making multiple runs necessary for reliable comparison. This variability was itself an interesting finding that we discuss further in Chapter 5.

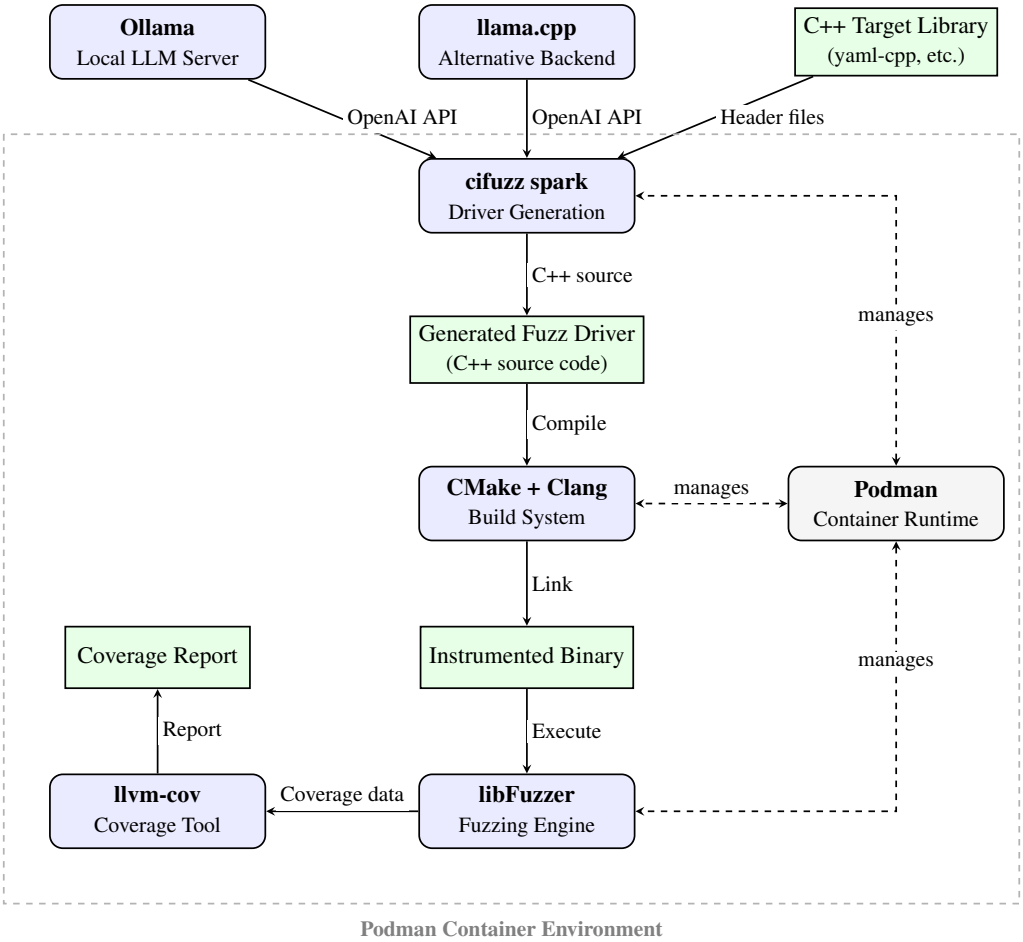

Figure 4.1: Evaluation Environment Component Diagram. Components (rounded boxes) interact via defined interfaces. Ollama or llama.cpp serve the LLM; cifuzz spark orchestrates driver generation; CMake/Clang builds the driver; libFuzzer executes it; llvm-cov measures coverage. All build and fuzzing components run inside a Podman container. The top-down flow shows the data transformation pipeline from source code to coverage report.

A full evaluation cycle for one model on one target took approximately 10 to 45 minutes, depending on model size and token generation speed (see Table 5.1 in Chap-

ter 5 for detailed timings). With 14 models and 6 primary targets, plus five runs per configuration for consistency checking, the complete Phase 1 evaluation required approximately two weeks of continuous testing.

The general fuzzing workflow (as introduced in earlier chapters) forms the foundation of our evaluation methodology, from defining targets from source code, generating fuzz tests via LLM, executing them through the fuzzing engine (libFuzzer with sanitizers), detecting crashes, and either reporting findings or mutating inputs for continued testing. Each evaluation cycle involved generating a driver, checking compilation, running the fuzzer for 60 seconds, measuring coverage with llvm-cov, and storing results.

4.3 Phase 2: Model Optimization with LoRA Fine-Tuning

After the initial evaluation showed promising results from Qwen 2.5-Coder models, we investigated whether fine-tuning could improve performance further. This phase addresses the optimization aspect of **RQ2**, asking whether smaller, fine-tuned models can outperform larger general-purpose models. The goal was to see if a small model, specifically trained for fuzz driver generation, could match or exceed larger ones.

4.3.1 Training Data Preparation

Most important for fine-tuning is the training data. High-quality examples of fuzz drivers paired with source code were essential. Previous work on LLM-based fuzzing like LLM4Fuzz [13] and TitanFuzz [11] has demonstrated that quality training data directly impacts generation success.

To build our dataset, we extracted training examples from OSS-Fuzz [12]. Google has fuzz-tested hundreds of open-source projects through OSS-Fuzz, and the fuzz drivers are publicly available. We focused on C and C++ drivers because those matched our target domain.

It is important to note that Google also maintains OSS-Fuzz-Gen [16], an automated system that uses LLMs to generate fuzz targets for OSS-Fuzz projects. While OSS-Fuzz-Gen pursues a similar high-level goal, namely automated fuzz driver creation, it differs from our approach in several important ways. OSS-Fuzz-Gen operates within Google's infrastructure, has access to complete build logs and project metadata, and targets breadth across hundreds of projects. Our approach instead focuses on depth for a specific domain (automotive C++ libraries), uses locally deployable open-source models rather than proprietary APIs, and integrates into enterprise CI/CD/CT pipelines with strict network isolation requirements. We used OSS-Fuzz as a *training data source* for fine-tuning, not as a competing tool.

Our extraction process involved four steps:

1. Clone the OSS-Fuzz repository
2. Identify projects with C/C++ fuzz drivers
3. For each driver, extract the header files it includes
4. Create input/output pairs: headers as input, driver as expected output

We created two datasets:

- **Small dataset**: 172 examples, focusing on quality over quantity
- **Extended dataset**: 709 examples, broader coverage but more noise

These examples included drivers for libraries like FreeType, LibPNG, SQLite, and zlib, covering a range of API styles from simple functions to complex object-oriented interfaces. We deliberately included diverse examples to help the model generalize rather than memorize specific patterns.

Representativeness for automotive applications. While these OSS-Fuzz libraries are not automotive-specific, they share key characteristics with automotive C/C++ codebases: they are written in C/C++, they expose public APIs with defined contracts, they process structured input data (configuration files, serialized data, media streams), and they must handle malformed inputs gracefully. Automotive ECU software similarly parses CAN/LIN messages, configuration files (YAML, JSON, XML), and sensor data streams. The fuzz driver patterns (consuming a byte buffer, translating it into API calls, catching exceptions) are structurally identical regardless of domain. We therefore consider these training examples representative of the *structural* task of fuzz driver generation, while acknowledging that they do not capture automotive-specific safety semantics (e.g., ISO 26262 ASIL-level constraints). The OSS-Fuzz corpus remains the largest publicly available source of validated C/C++ fuzz drivers; no comparable automotive-specific corpus exists at this time.

Quality control was important. We manually reviewed a sample of the extracted drivers to verify they were functional and not stub implementations. Approximately 15% (106 out of the initial pool) of extracted drivers were discarded because they were incomplete or contained obvious errors.

4.3.2 LoRA Configuration and Training

We used Low-Rank Adaptation (LoRA) [20] rather than full fine-tuning. Full fine-tuning updates all model weights, which for a 7B-parameter model requires multiple high-end GPUs and days of training. LoRA adds small trainable matrices alongside

the frozen base model. For our 1.5B model, this reduced the number of trainable parameters from 1.5 billion to approximately 4.7 million (a 99.7% reduction), enabling training on a single consumer GPU while achieving similar results on specialized tasks.

Our LoRA configuration:

```
# LoRA Configuration
LORA_RANK = 16            # Rank controls adaptation complexity
LORA_ALPHA = 32           # Alpha is the scaling factor
DROPOUT = 0.1             # Prevents overfitting
TARGET_MODULES = ["q_proj", "v_proj", "k_proj", "o_proj"]
DEVICE = "auto"           # Uses optimal hardware
DTYPE = "float16"         # Memory-efficient precision
```

Listing 4.6: LoRA Configuration Parameters

A rank of 16 was a balance between adaptation capacity and training speed. Higher ranks can capture more complex patterns but require more memory and training time. We tested ranks of 8, 16, and 32. Rank 16 produced the strongest output for our dataset size.

We trained on the Qwen 2.5-Coder 1.5B base model. This is a small model that runs on a single consumer GPU with 8 GB VRAM (e.g., Apple M1 Pro or NVIDIA RTX 3070). This size was chosen to investigate whether fine-tuning could make small models competitive with larger ones.

Training used standard supervised fine-tuning:

- Input: Header files and fuzzing instructions
- Target: Complete fuzz driver
- Learning rate: 2e-5 with cosine decay
- Epochs: 3
- Batch size: 4 (with gradient accumulation)

4.3.3 Fine-Tuned Model Evaluation

After training, we evaluated the fine-tuned models on the same yaml-cpp benchmark used in Phase 1. The results showed measurable efficiency improvements, summarized with absolute numbers in Table 4.2.

Coverage of the fine-tuned 1.5B models remained comparable to the larger Qwen 2.5-Coder 32B model, which achieved 43% line coverage (see Chapter 5 for detailed coverage results). The reference row shows that while the 32B model achieves higher coverage, it requires over three times the generation time.

Table 4.2: Fine-Tuned Model Evaluation Results on yaml-cpp (Absolute Numbers)

Model Version	Gen. Time	Tokens Used	Time Saved	Tokens Saved
Base Qwen 2.5-Coder 1.5B	15 min	112 k	(baseline)	(baseline)
LoRA (172 examples)	12 min	65 k	3 min (20%)	47 k (42%)
LoRA (709 examples)	10 min	50 k	5 min (33%)	62 k (55%)
Qwen 2.5-Coder 32B (ref.)	33 min	45 k	(reference)	(reference)

Fine-tuned models generated more concise code. They learned the structure of fuzz drivers and did not waste tokens on unnecessary explanations or boilerplate. These efficiency gains meant that generating each driver took less time and fewer API tokens, which directly impacts operational costs.

More training data did not always produce proportionally better results. The 709-example dataset achieved the best efficiency improvements (see Chapter 5 for detailed metrics), but it included some lower-quality drivers that may have introduced noise. While the extended dataset reduced generation time and token usage, the coverage quality did not improve proportionally. Quality matters more than quantity for fine-tuning specialized tasks.

We also tested the fine-tuned models on RapidJSON and pugixml to check generalization. Performance was similar to yaml-cpp, suggesting the fine-tuning improved general fuzz driver generation rather than overfitting to specific libraries.

4.4 Phase 3: Enterprise CI/CD/CT Integration

Phase 3 focused on deploying the validated approach from Phases 1 and 2 into CARIAD's production CI/CD/CT infrastructure using Azure OpenAI [28] as the LLM backend. The central question for this phase was **RQ3**: Can LLM-assisted fuzz driver generation be integrated into secure enterprise CI/CD/CT pipelines while meeting performance, cost, and security requirements? The transition from controlled local experiments with Ollama [33] to enterprise deployment revealed additional infrastructure considerations that constituted the primary engineering focus of this phase.

To guide the integration, we derived the following concrete requirements from CARIAD's operational context:

- **Performance requirements:**
  1. Driver generation must complete within the CI/CD/CT feedback window (targeted at 15 to 30 minutes, with up to 45 minutes acceptable for complex libraries).

2. Fuzzing execution must not block the main build pipeline; it may run asynchronously.
3. The system must support at least 10 target libraries per day without manual intervention.

- **Cost requirements:**
  1. Annual infrastructure cost must remain below €2,000 for moderate usage.
  2. The solution must not require dedicated GPU hardware on the CI/CD/CT runners.
  3. Token consumption must be predictable and auditable.
- **Security requirements:**
  1. No proprietary source code may leave the corporate network perimeter via unencrypted or unapproved channels.
  2. LLM API access must comply with the corporate zero-trust network model.
  3. Generated code must be sandboxed (containerized) and instrumented with sanitizers before execution.
  4. API credentials must be stored in approved secret management systems (GitHub Actions secrets).

The architectural challenges described below arose directly from these requirements, and the solutions we developed were evaluated against them.

4.4.1 Architectural Challenges

Our initial integration strategy appeared simple in concept. The working local pipeline used standard Linux tools, and enterprise runners operated in similar environments. However, the enterprise network security policies required careful architectural planning to maintain compliance.

Challenge 1: Network Isolation

CARIAD operates with a zero-trust security model. CI/CD/CT runners are isolated from the public internet, and external connections require explicit approval. Integrating cloud-based AI services within this security framework required a tailored architectural approach.

Our initial pipeline configuration was designed for direct connectivity and was subsequently adapted for the network isolation requirements. The Azure OpenAI requests were routed through the corporate firewall, which, as expected in a zero-trust environment, did not permit direct external API access.

Important clarification regarding data sent to Azure OpenAI. The prompts sent to the LLM contain only *public header files* (function signatures, type definitions, class declarations) extracted by cifuzz spark, combined with generic fuzzing instructions. They do *not* contain proprietary source code implementations, business logic, or internal documentation. The header files describe the public API surface of the target library, which is equivalent to information already visible in publicly distributed SDK packages. For open-source libraries like yaml-cpp, this information is entirely public. For internal CARIAD libraries, only the public interface declarations are transmitted, not the implementation files. This distinction was a key factor in the security review that approved the architecture.

The resulting error message required investigation to diagnose:

```
Error: fatal: unable to access 'https://<corporate-git>.example.com/...':
Failed to connect to 127.0.0.1 port 9000 after 0 ms: Couldn't connect to server
```

Investigation revealed that the proxy configuration scope needed refinement to separate Azure-bound traffic from internal Git operations. The runner was directing Git traffic through the Azure proxy endpoint, which was configured exclusively for LLM API access.

Challenge 2: Container Networking and Security Model

Runners use Buildah containers for isolation. Each build step runs in its own network namespace, providing strong security boundaries. External service access must be explicitly configured at the container level.

Attack model. The container isolation addresses two distinct threat categories. First, *tenant isolation*: in a shared runner environment, containers prevent one CI/CD/CT job from accessing another's data or network connections. Buildah's rootless, daemonless architecture strengthens this boundary compared to Docker, as there is no long-running daemon with elevated privileges that could be exploited. Second, *generated code containment*: since LLM-generated fuzz drivers are untrusted code, they execute inside containers with restricted privileges, no shell access, and sanitizer instrumentation (ASan/UBSan) that terminates on memory violations.

However, container isolation does *not* protect against a compromised cloud provider. If the Azure infrastructure itself were malicious, it could inspect the prompts and responses. This is a fundamental limitation of any cloud-based approach and is mitigated through contractual and compliance mechanisms (Azure's SOC 2, ISO 27001, and GDPR certifications) rather than technical isolation alone. For organizations requiring protection against cloud-provider-level threats, the alternative is on-premises model deployment, which we evaluated in Phase 1 using Ollama (Section 4.2).

We tried several approaches:

Host network mode:

```
buildah run --network host -v $(pwd):/project $containerid /bin/
    bash -c "cifuzz spark target"
```

This provided the container with host-level network access, though the firewall policy continued to govern external connections.

Custom DNS entries:

```
buildah run --add-host <endpoint>.openai.azure.com:${OPENAI_IP}
    ...
```

This approach was not effective because the connectivity requirement originated from firewall policy rather than DNS resolution.

Challenge 3: Credential Management

Even with network connectivity established, API keys needed to be securely passed to the container. GitHub Actions secrets provided the storage, but injecting them into Buildah containers required explicit environment variable passing:

```
buildah run \
  --env CIFUZZ_LLM_API_URL \
  --env CIFUZZ_LLM_AZURE_DEPLOYMENT_NAME \
  --env CIFUZZ_LLM_MAX_TOKENS \
  --env CIFUZZ_LLM_API_TOKEN \
  -v $(pwd):/$PROJECT_FOLDER ${{ env.containerid }} \
  /usr/bin/bash -c "cd /project/build && cifuzz spark
    target_name"
```

Each environment variable had to be passed explicitly. Careful validation of the configuration was necessary, as missing LLM credentials would cause the generation step to be skipped without halting the pipeline. We addressed this by adding pre-flight checks to verify all required variables before the LLM call.

4.4.2 Self-Hosted Runner Solution

Following our investigation, we determined that direct Azure OpenAI access required a network architecture aligned with the enterprise security model. We collaborated with the infrastructure team to implement an appropriate solution.

Azure Private Link Architecture

Our approved solution used Azure Private Link [29]. This creates a private endpoint for Azure services that appears inside the corporate network. Traffic to the LLM endpoint never crosses the public internet.

From the runner's perspective, the Azure OpenAI endpoint has an internal IP address. The firewall allows internal traffic, so the LLM calls succeed.

Azure trust within the zero-trust model. A natural question arises: if CARIAD operates a zero-trust network, why is Azure considered trusted? The answer is that zero-trust refers to the *network perimeter*: no device or connection is trusted by default based solely on network location. Azure Private Link integrates Azure services

into the corporate network as a *private endpoint*, meaning the traffic never traverses the public internet and is subject to the same network security group rules as internal services. Trust in Azure as a *service provider* is established through a separate mechanism: contractual agreements (Microsoft Enterprise Agreement), compliance certifications (SOC 2 Type II, ISO 27001, C5), and data processing agreements under GDPR. The zero-trust model governs network access paths; provider trust is established through governance and compliance frameworks. This layered approach, combining a private network path with contractual trust, satisfies CARIAD's security requirements without contradicting the zero-trust principle.

Setting up Private Link involved coordination with the infrastructure team, following the organization's established approval workflow:

1. Request submission with business justification
2. Security review of the use case
3. Azure subscription configuration
4. DNS updates for private endpoint resolution
5. Network security group rules
6. Testing and validation

We prepared a detailed business case including cost analysis (approximately €74 to €1,452 annually depending on usage) and security risk assessment. This documentation supported the approval process.

Working Pipeline Configuration

Once Private Link was operational, the pipeline worked reliably. Figure 4.2 illustrates the CI/CD/CT pipeline as a workflow diagram. The full YAML configuration is provided in Appendix A.4.

Our pipeline includes corpus persistence through JFrog Artifactory. Each run downloads the previous corpus, runs fuzzing to extend it, and uploads the updated corpus. This way, coverage improvements accumulate over time rather than starting fresh each build.

The network architecture required for this deployment is illustrated in Figure 6.1 (Chapter 6). As discussed in the subsequent Discussion chapter, the Azure Private Link setup enables secure communication between the CARIAD internal network and Azure OpenAI services without routing traffic through the public internet, while maintaining full security compliance. This architectural solution was the key component for enterprise deployment.

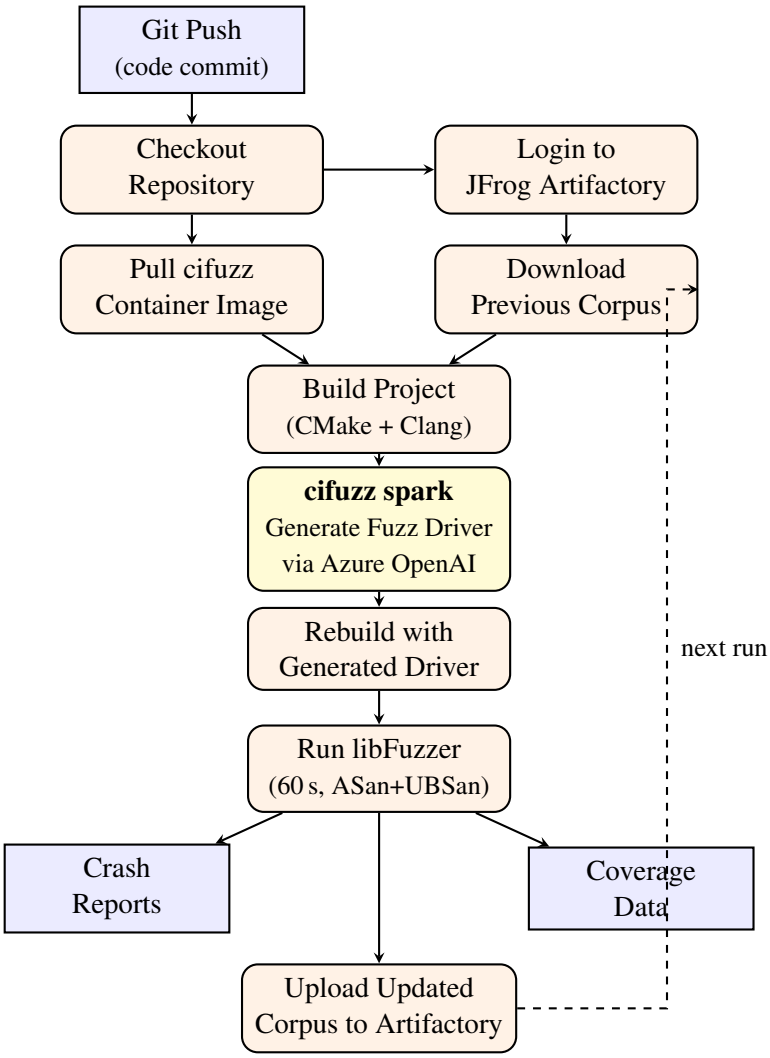

Figure 4.2: CI/CD/CT Pipeline Workflow. Processing steps are in rounded boxes; data artifacts in rectangular boxes. The highlighted step (cifuzz spark) invokes the LLM via Azure Private Link. Corpus persistence through JFrog Artifactory enables cumulative coverage improvement across runs.

4.4.3 Operational Considerations

With the technical implementation complete, several operational concerns remained.

Cost Management

Azure OpenAI charges per token. Each fuzz driver generation uses approximately 3000 to 8000 tokens depending on the target complexity. We calculated costs for different usage scenarios:

Table 4.3: Azure OpenAI Cost Projections for Different Usage Scenarios

Scenario	Targets/Day	Builds/Day	Annual Tokens	Annual Cost
Light	5	1	3.1M	€74
Moderate	10	2	12.5M	€219
Heavy	25	5	78M	€726
Enterprise	50	10	156M	€1,452

For context, even the enterprise scenario (€1,452 per year) is approximately two orders of magnitude below the cost of a single full-time security engineer (approximately €80,000 to €120,000 annually at industry rates). However, the LLM-generated drivers supplement rather than replace expert security engineering work, so this comparison captures only the infrastructure cost component. A detailed economic analysis including human oversight costs appears in Chapter 5.

Failure Handling

LLM outputs are non-deterministic. Sometimes the generated driver does not compile. Sometimes it compiles but crashes immediately. The pipeline was designed to handle these cases gracefully.

We added fallback logic, so if the LLM-generated driver fails, the pipeline falls back to any existing manually-written drivers. The build does not fail due to unsuccessful AI generation. Generated drivers that work are kept. Those that fail are logged for review but do not block the pipeline.

Security Review

Generated code running in CI/CD/CT pipelines poses security considerations. What if the LLM generates malicious code? We addressed this concern in our security review:

1. The code runs inside containers with limited privileges
2. ASan and UBSan catch memory-related errors and undefined behavior during execution
3. Generated code is compiled and executed, not given shell access
4. The LLM only sees public header files, not sensitive source code

5. All generated code is logged for audit purposes

After reviewing these controls, the security team approved the architecture. They requested additional monitoring of token usage and periodic audits of generated code quality.

The next chapter presents experimental results in detail, with quantitative analysis of model performance and coverage metrics across all evaluated configurations.

5 Experimental Results

Our three-phase research, conducted between May and September 2025 with infrastructure integration running in parallel with the initial local experiments, produced both expected and unexpected results. Some models generated working fuzz drivers while the majority did not. Fine-tuning improved efficiency but not coverage. Enterprise deployment was technically feasible but required more infrastructure work than anticipated. This chapter presents these findings in detail, organized by research phase, and maps them to the research questions from Chapter 1.

5.1 Experimental Setup

5.1.1 Target Selection and Criteria

Choosing appropriate target libraries required careful consideration. We needed C++ libraries because safety-critical automotive systems running on AUTOSAR (AUTomotive Open System ARchitecture) [2] platforms are predominantly written in C or C++. AUTOSAR is the dominant software architecture standard for automotive electronic control units (ECUs), making C++ relevance essential for practical automotive application. While Python-based testing would have involved less complexity, C++ was selected to match the language used in safety-critical automotive systems at CARIAD.

After evaluating several candidates, we selected yaml-cpp as our primary target. This library parses YAML configuration files, a common task in automotive software where configuration management matters significantly. YAML is widely used in build systems, CI/CD/CT pipelines, and deployment configurations across the automotive industry. The yaml-cpp library is found in several public repositories within automotive organizations including CARIAD. The library contains 35 source files with 1,061 potential fuzzing candidates identified through cifuzz spark analysis. It is also well-documented and actively maintained. OSS-Fuzz [12] already covers it, which provided a baseline for comparison. We could measure whether our AI-generated drivers matched, exceeded, or fell short of existing human-written ones.

Additional targets included pugixml, jsoncons, fmt, spdlog, and glm. These are widely used open-source C++ libraries that also appear in public repositories of automotive companies, including CARIAD's internal and public codebases. While they are not automotive-specific, they represent the types of general-purpose C++ depen-

dencies commonly integrated into automotive software stacks for tasks such as XML/JSON configuration parsing (pugixml, jsoncons), log management (spdlog), string formatting (fmt), and mathematical computations for sensor processing (glm). These provided broader validation across different library types, and results for the top-performing models on these libraries are presented in Section 5.2.3.

5.1.2 Hardware and Software Configuration

All local experiments ran on macOS with an Apple M1 Pro processor. The container runtime used Podman with 4 CPUs and 8 GB memory allocated. This choice was deliberate. We wanted hardware that typical automotive development teams have access to. If our approach required dedicated GPU clusters, practical adoption would be more constrained.

Our software stack consisted of Podman for containers (chosen to match the Linux-based container runtime used in the enterprise CI/CD/CT environment), CMake with Clang for building, libFuzzer via cifuzz for fuzzing, and Ollama for local model inference. For enterprise deployment testing, we used Azure OpenAI with GPT-4o connected via Azure Private Link, a configuration established through the enterprise provisioning process, as detailed in Chapter 4.

5.2 LLM Fuzz Driver Generation Results

5.2.1 Successful Models: Performance Data

We evaluated models across different size categories on the yaml-cpp repository. The results, summarized in Table 5.1, revealed several unexpected patterns.

Table 5.1: Model Performance Metrics on yaml-cpp Repository

Model	Code Coverage	Time Taken	Tokens Used	Unique Test Cases	Successful Tests
Qwen 2.5-Coder 32B	43.08%	32m 57s	45.1k	2,040	2
Gemma 3 27B	45.06%	33m 33s	40.2k	2,050	2
Phi-4 14B	34.26%	36m 36s	71.5k	2,220	1

The “Successful Tests” column requires explanation, as this metric counts tests that discovered actual bugs or edge cases, not tests that simply compiled and ran. Most generated tests exercised valid code paths without finding vulnerabilities, which is expected behavior, not failure. The high number of unique test cases with few “successful” findings reflects that fuzzing is a volume-based process where many tests are generated with the expectation that a subset will discover issues. It is important to note that all three models listed in Table 5.1 produced drivers that compiled and ran successfully. The remaining eleven models (discussed in Section 5.2.2) failed at the

generation stage: their outputs either did not compile due to syntax errors, missing includes, or incorrect API usage, or crashed immediately at runtime due to invalid function calls. These models are therefore not included in Table 5.1 because they did not reach the fuzzing stage.

Line coverage measures the percentage of source code lines executed during testing. It is the standard metric we use throughout this thesis for evaluating fuzz driver quality (measured via `llvm-cov`). Gemma 3 27B achieved the highest coverage at 45.06%, slightly outperforming the larger Qwen 2.5-Coder 32B, which reached 43.08%. This was unexpected. The smaller model outperformed the larger one while using fewer tokens. We initially suspected measurement error, but the results held across multiple runs with consistent reproducibility.

That said, these numbers require context. While 45% coverage represents a meaningful automated achievement, we examined several comparable manually-written fuzz drivers in the OSS-Fuzz corpus for yaml-cpp and similar parsing libraries (pugixml, RapidJSON, libxml2). These expert-crafted drivers typically achieve 60 to 80% line coverage, based on coverage reports available through the OSS-Fuzz project dashboards[1]. Our best AI-generated driver (Gemma 3 at 45.06%) reaches approximately 56 to 75% of the expert-level range, falling 15 to 35 percentage points short. The value proposition is not matching expert coverage levels but automating fuzz driver creation, a task that would otherwise require 2 to 8 hours of manual engineering per target (see Section 5.4.2).

Phi-4 14B showed lower coverage at 34.26% despite generating more unique test cases (2,220 vs 2,050). It required significantly more tokens, 71.5k compared to Gemma's 40.2k, making it considerably less efficient for production use. More test cases do not necessarily mean better coverage.

The comparative performance of these three successful models is visualized in Figure 5.1. Gemma 3 27B and Qwen 2.5-Coder 32B achieved the highest coverage levels (45% and 43% respectively), while Phi-4 14B reached 34%. Only these three of the fourteen evaluated models produced functional fuzz drivers that reached the fuzzing stage. The remaining eleven models did not produce functional output at the code generation stage and are discussed in Section 5.2.2.

5.2.2 Unsuccessful Models: Critical Findings

Several models did not produce usable output, as summarized in Table 5.2. This was one of the more important findings of the research.

The majority of models failed at the compilation stage, producing code with syntax errors, incorrect API usage, or incomplete driver structures. Yi 34B was a notable exception: it produced code that compiled in some cases but achieved zero percent coverage at runtime. The generated drivers were syntactically valid but did not ex-

[1] https://introspector.oss-fuzz.com/

Figure 5.1: Code Coverage by Model (yaml-cpp). Only three of the fourteen evaluated models produced functional fuzz drivers.

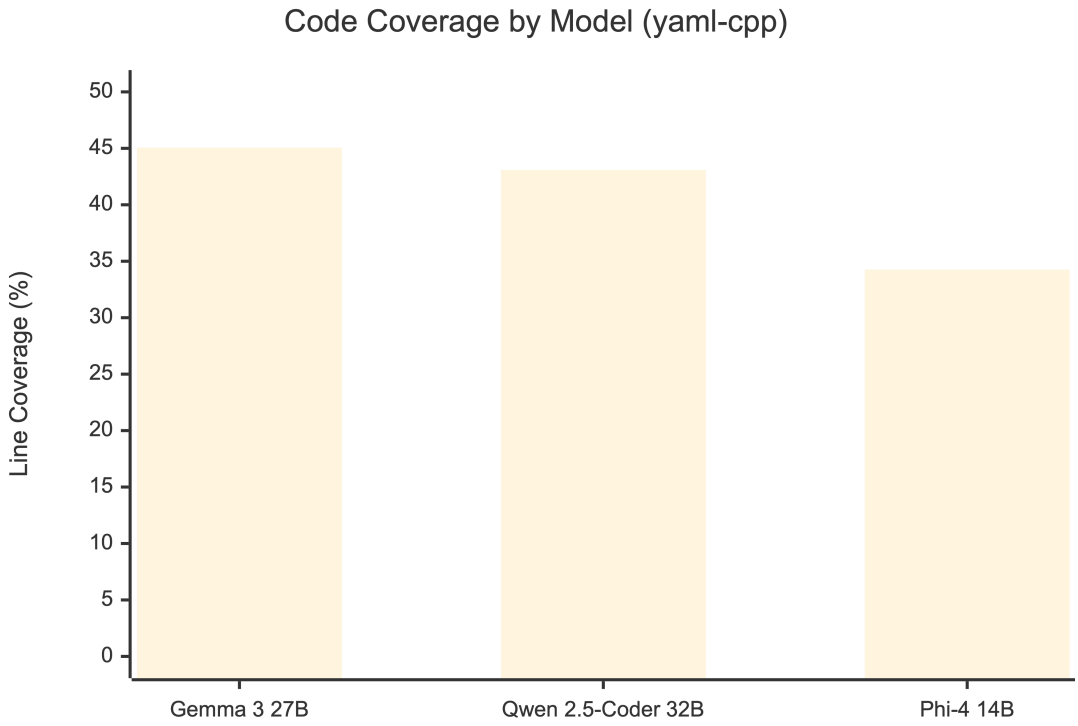

Table 5.2: Unsuccessful Models and Failure Analysis on yaml-cpp

Model	Parameters	Failure Stage	Failure Reason
Qwen 2.5-Coder 14B	14B	Compilation	Incorrect API usage, missing includes
Qwen 2.5-Coder 7B	7B	Compilation	Incomplete driver structure
Qwen 2.5-Coder 1.5B	1.5B	Compilation	Insufficient context understanding
DeepSeek Coder 33B	33B	Compilation	Wrong function signatures
DeepSeek Coder V2 Lite	16B	Compilation	Incomplete driver output
CodeLlama 34B Instruct	34B	Compilation	Structural errors in generated code
WizardCoder 34B	34B	Compilation	Malformed driver structure
Yi 34B	34B	Runtime	Generation hallucinations, 0% coverage
Gemma 3 9B	9B	Compilation	Insufficient model capacity
Gemma 3 4B	4B	Compilation	Insufficient model capacity
Mixtral 46.7B	46.7B	Infrastructure	Model exceeded available hardware capacity

ercise the target library effectively. Functions were called with incorrect parameter combinations, or the driver simply returned immediately without exercising any library functionality. We tested variations of the prompt template to verify this was not a prompt engineering issue, and selected the version that most consistently produced compilable output. The results were consistent across attempts.

We also tested DeepSeek R1, a reasoning-focused model, outside the original 14-model benchmark. It was designed for reasoning tasks, but in our experiments, that capability did not translate to effective fuzz driver generation. Mixtral's 46.7 billion parameters exceeded the available hardware capacity, preventing completion of a full evaluation run.

A clear pattern emerged from Table 5.2. The successful models (Qwen 2.5-Coder 32B, Gemma 3 27B, Phi-4 14B) share recent architectures and high-quality training data, but they span both code-specialized and general-purpose categories. Size alone does not determine quality for specialized tasks like fuzz driver generation. Furthermore, even among code-specialized models, only those with the right combination of architecture and training data (such as Qwen 2.5-Coder 32B) succeeded, while others (CodeLlama 34B, WizardCoder 34B) failed despite their code-focused training.

5.2.3 Cross-Library Validation

To validate whether the yaml-cpp findings generalize to other libraries, we tested the top-performing models on additional C++ targets. Not all models were evaluated on every library due to time constraints, so we focused on the models that showed the most promise.

fmt (formatting library):

Table 5.3 presents the results for the fmt library. In addition to the 14 models from our primary benchmark, we tested Llama 3 and Qwen 1.5 7B on fmt to broaden coverage across model families.

Table 5.3: Model Performance on fmt Library

Model	Code Coverage	Time Taken	Tokens Used	Result
Phi-4 14B	100%	12m 8s	59.8k	Successful
Llama 3	100%	3m 41s	27.6k	Successful
Qwen 1.5 7B	89.47%	6m 9s	50k	Successful
Qwen 2.5-Coder 32B	0%	43m 20s	59.8k	Unsuccessful
Gemma 3 27B	0%	41m 59s	73.9k	Unsuccessful

Table 5.3 presents results that are counterintuitive compared to the findings from Section 5.2.1. The two best-performing models on yaml-cpp (Qwen 2.5-Coder 32B and Gemma 3 27B) both achieved 0% on fmt, while smaller models achieved near-perfect coverage.

We examined the generated outputs to understand this reversal. The fmt library exposes a small, well-documented API surface with clear input/output contracts (e.g., `fmt::format(format_string, args...)`). The smaller models generated concise drivers that directly called these formatting functions with the fuzzer-provided input. In contrast, the larger models generated verbose, over-engineered drivers that attempted to construct complex formatting scenarios but introduced compilation errors or invalid API sequences in the process. Specifically, Qwen 2.5-Coder 32B generated drivers that referenced internal fmt implementation details not exposed in the public headers, while Gemma 3 27B produced drivers that called deprecated or non-existent overloads.

This observation is consistent with findings by Zhang et al. [45], who reported that LLM-generated fuzz drivers for simpler APIs achieve higher success rates, and that larger models do not uniformly outperform smaller ones across different target complexities. The key factor appears to be the match between model output verbosity and API complexity: simpler APIs benefit from concise, targeted generation, while complex APIs like yaml-cpp require the broader context understanding that larger models provide. This finding reinforces that no single model dominates across all library types, and model selection should consider the target API characteristics.

pugixml (XML processing library):

The results for fuzz-testing code from the pugixml library are presented in Table 5.4.

Table 5.4: Model Performance on pugixml Library

Model	Code Coverage	Time Taken	Tokens Used	Result
Qwen 2.5-Coder 32B	34.77%	37m 24s	43.1k	Successful
Gemma 3 27B	0%	1h 12m 8s	122k	Unsuccessful

On pugixml, Qwen 2.5-Coder 32B achieved 34.77% coverage, comparable to its yaml-cpp performance (43.08%). Gemma 3 27B did not achieve any coverage despite consuming considerably more tokens (122k vs 40.2k on yaml-cpp), suggesting the XML parsing API posed challenges that the model was not able to address in this configuration.

jsoncons, glm, and spdlog (Qwen 2.5-Coder 32B only):

Table 5.5 summarizes the results for testing jsoncons, glm, and spdlog.

As shown in Table 5.5, jsoncons showed the highest coverage of any library we tested (45.64%), slightly exceeding even yaml-cpp. Both are parsing libraries with similar API patterns, which may explain the consistency. glm (a mathematics library for graphics) and spdlog (a logging library) both produced 0% coverage, indicating that the models did not generate effective drivers for non-parsing API patterns. Libraries with mathematical or configuration-heavy interfaces appear more challenging for current LLMs.

Table 5.5: Qwen 2.5-Coder 32B Performance Across Additional Libraries

Library	Code Coverage	Time Taken	Tokens Used	Test Cases	Successful Tests
jsoncons	45.64%	37m 24s	43.1k	2,500	1
glm	0%	42m 55s	56.8k	0	0
spdlog	0%	49m 33s	62.9k	0	0

Cross-library summary: The results show that model performance varies considerably across libraries. Parsing libraries (yaml-cpp, jsoncons, pugixml) consistently produced the highest coverage, while utility libraries (glm, spdlog) were more challenging. Library complexity did not always predict difficulty, as fmt's simpler API yielded perfect coverage with smaller models. These findings suggest that API style and documentation quality matter more than library size or domain.

5.3 Model Optimization Results

Having established that model architecture and training quality determine success more than size or specialization label, we investigated whether fine-tuning could further improve efficiency while reducing computational requirements. These experiments address the optimization aspect of **RQ2**: can smaller models with domain-specific training match or exceed larger general-purpose models?

5.3.1 LoRA Fine-Tuning Efficiency

Based on Phase 1 results, we selected the Qwen 2.5-Coder 1.5B model for fine-tuning experiments. The goal was to determine whether a small, fine-tuned model could match larger models while using markedly fewer resources.

We applied Low-Rank Adaptation (LoRA) [20] fine-tuning using fuzz drivers from Google's OSS-Fuzz project as training data. LoRA works by freezing most of the original model weights and inserting small trainable matrices into the attention layers, so only a fraction of parameters are updated during training. This makes fine-tuning feasible on consumer hardware without the memory requirements of full model training. Our approach builds on recent work in LLM-based fuzzing [13, 11] but applies more aggressive model compression through fine-tuning. We curated two training datasets from publicly available OSS-Fuzz repositories, a small one with 172 examples and an extended one with 709 examples. Each example consisted of a fuzz driver paired with its target API context, covering security vulnerability patterns, fuzzing techniques, and automotive-specific code patterns.

Our fine-tuning configuration:

- LoRA Rank: 16

- LoRA Alpha: 32
- Dropout: 0.1
- Target Modules: q_proj, v_proj, k_proj, o_proj
- Precision: float16

Results showed measurable efficiency improvements:

Table 5.6: LoRA Fine-Tuning Efficiency Improvements

Model Version	Time Taken	Tokens Used	Efficiency Improvement
Base 1.5B	15 min	112k	Baseline
172 examples	12 min	65k	20% faster, 42% fewer tokens
709 examples	10 min	50k	33% faster, 55% fewer tokens

With 709 examples, the extended dataset produced the largest improvements with 33% faster generation and 55% fewer tokens compared to the base model. The fine-tuned model learned to generate more concise, targeted code rather than verbose outputs padded with unnecessary boilerplate.

5.3.2 Comparative Analysis

Training data quality and quantity both matter, though quality appears more important. The 709-example dataset outperformed the 172-example dataset, but the improvement was incremental rather than dramatic. More diverse examples help the model generalize, but there are diminishing returns.

One key insight is that a fine-tuned small model can match larger models while using markedly fewer computational resources. This makes enterprise deployment practical, as it does not require dedicated GPU infrastructure (typically $10,000+ per GPU unit) to run effective fuzz driver generation.

However, we should note the limitations. The fine-tuned 1.5B model still does not match the coverage achieved by the larger Qwen 32B or Gemma 27B models. What it offers is efficiency, acceptable coverage at much lower cost and resource requirements.

The efficiency gains from fine-tuning directly impact deployment feasibility in resource-constrained environments. However, efficiency alone does not determine practical viability, cost considerations also play a major role in enterprise adoption.

5.4 Economic Analysis and Resource Metrics

Beyond technical performance, successful enterprise deployment requires favorable economics. This section addresses the cost component of **RQ3**: Can LLM-assisted fuzz driver generation be integrated into secure enterprise CI/CD/CT pipelines while meeting performance, cost, and security requirements? As defined in Chapter 4 (Section 4.4), the performance requirements include completing driver generation within the 15 to 30 minute CI/CD/CT feedback window, the cost requirements cap annual infrastructure spending below €2,000, and the security requirements mandate that no proprietary source code leaves the corporate network. We analyzed both direct infrastructure costs and opportunity costs relative to manual approaches.

We analyzed costs in detail because the question is not only "does this work?" but "is this economically viable for real deployment?"

5.4.1 Azure OpenAI Cost Projections

Based on Azure OpenAI pricing as of July 2025 (the time of our experiments):

Table 5.7: Azure OpenAI Cost Projections for Different Usage Levels

Usage Level	Daily Cost	Monthly Cost	Annual Cost	Use Case
Light	€0.28	€6.16	€73.92	Development/testing
Moderate	€0.83	€18.26	€219.12	Small team production
Heavy	€2.75	€60.50	€726.00	Full enterprise deployment
Enterprise	€5.50	€121.00	€1,452.00	Complete automation

Even at the enterprise level with complete automation, annual costs stay under €1,500 (Table 5.7), which is less than 2% of a single security engineer's annual salary (approximately €80,000 to €120,000 at industry rates). To put this in perspective: the enterprise-level annual cost of €1,452 is comparable to two days of engineering labor, yet the automated pipeline can generate and evaluate drivers continuously throughout the year. However, this represents only the direct infrastructure costs.

5.4.2 Comparison with Manual Testing

Industry estimates for manual fuzz driver development:

- Senior security engineer hourly rate: €80 to €120 per hour (based on published industry benchmarks for Western European markets)

- Time to write an effective fuzz driver: 2 to 8 hours per target (lower end for well-documented parsing APIs, upper end for complex C++ libraries with heavy template usage)
- Annual cost for security engineering resources: varies by market and seniority

The AI-driven approach offers:

- Infrastructure cost: €219-€1,452 annually
- Estimated time savings: 75 to 94% reduction in initial driver creation time (based on comparison of approximately 30 minutes automated generation vs. 2 to 8 hours of manual writing per target)

The economic advantage appears clear, but there is an important caveat. Current LLMs do not incorporate domain-specific automotive safety requirements, do not account for ISO 26262 compliance considerations, and may occasionally generate tests that are syntactically correct but lack semantic depth. It should be noted that none of the test cases in our evaluation incorporated safety-related constraints (e.g., ASIL levels, timing requirements, or fault-tolerance checks); all generated drivers focused exclusively on functional correctness and coverage of the public API surface. Human oversight remains essential. The time savings estimate assumes engineers spend their time reviewing and improving AI-generated drivers rather than writing from scratch, not that AI fully replaces human judgment. The cost of this human oversight is not captured in the infrastructure costs above, and the actual savings will vary depending on the complexity of the target library and the quality of the generated driver.

5.5 Summary

The experiments produced several clear findings, along with some important caveats.

Model architecture and training quality matter more than size alone. Gemma 3 27B outperformed Yi 34B despite having fewer parameters. However, the relationship between model type and success is nuanced: among the three successful models, Qwen 2.5-Coder 32B is explicitly code-specialized, while Gemma 3 27B and Phi-4 14B are general-purpose models with strong code capabilities. Conversely, several explicitly code-specialized models (CodeLlama 34B, WizardCoder 34B, DeepSeek Coder 33B) failed entirely. This suggests that neither size nor a code-specialization label alone determines fuzz driver quality; rather, the specific training data quality and model architecture are decisive.

Performance also varies across library types. Parsing libraries (yaml-cpp, jsoncons, pugixml) consistently produced the best coverage results, while utility libraries (glm, spdlog) proved more challenging. The fmt library revealed that smaller models

can outperform larger ones on simpler APIs, with Phi-4 14B and Llama 3 achieving 100% coverage where the larger Qwen 2.5-Coder 32B and Gemma 3 27B both achieved 0%.

Fine-tuning enables efficient deployment. The fine-tuned 1.5B model achieved 33% faster generation and 55% fewer tokens compared to the base model, though it still did not match the coverage of the larger successful models (Qwen 2.5-Coder 32B, Gemma 3 27B).

The economics appear favorable as well. Annual infrastructure costs range from €74 for light usage to €1,452 for full enterprise deployment. Manual security engineering involves higher personnel investment, though the two approaches serve complementary rather than interchangeable functions. The LLM automates initial driver creation while human expertise remains necessary for review and refinement.

Mapping these findings to our research questions:

RQ1 (Effectiveness): Can Large Language Models generate fuzz drivers for C++ code that compile successfully, execute without errors, and achieve meaningful code coverage? The results support this at least to some extent. Our best models achieved over 40% coverage on yaml-cpp and up to 100% on fmt, with drivers that compiled and executed correctly. Cross-library testing (Section 5.2.3) confirmed these results generalize to other parsing libraries like jsoncons (45.64%) and pugixml (34.77%), though performance on non-parsing libraries was more variable. We consider 34 to 45% coverage "meaningful" because it represents automated baseline coverage that would otherwise require 2 to 8 hours of manual engineering per target. While this falls short of expert human-written drivers (60 to 80% coverage), the value lies in automation at scale, not matching human experts.

RQ2 (Optimization): Does model size determine fuzz driver quality, or can smaller models with domain-specific training match or exceed larger general-purpose models? Size does not determine quality. The three successful models (Qwen 2.5-Coder 32B, Gemma 3 27B, Phi-4 14B) span different sizes and categories. Notably, several larger code-specialized models (CodeLlama 34B, WizardCoder 34B) failed entirely. Fine-tuning further improved efficiency, though not coverage.

RQ3 (Feasibility): Can LLM-assisted fuzz driver generation be integrated into secure enterprise CI/CD/CT pipelines while meeting performance, cost, and security requirements? Integration proved feasible at costs under €1,500 annually, but the process required careful infrastructure planning (Azure Private Link) to meet the security requirements of regulated environments.

6 Discussion and Conclusion

This chapter interprets the experimental results from Chapter 5, connects them to the research questions posed in Chapter 1, and examines what the findings mean for practitioners. We discuss both successes and failures, then close with recommendations for future research and a summary of contributions.

6.1 Interpretation of Results

6.1.1 What the Data Reveals

The experimental results revealed patterns we did not anticipate at the outset of this work.

Most surprising was the inverse relationship between model size and performance. Conventional wisdom in machine learning suggests larger models perform better. Our fuzz driver experiments showed something different.

Why did the larger general-purpose models fail while smaller specialized ones succeeded? We analyzed the generated code outputs and identified a consistent pattern. The larger models generated code that appeared syntactically plausible but did not align with the structural requirements of the fuzz driver task. They would create elaborate test scaffolding, call APIs in reasonable sequences, but miss the essential structure that a libFuzzer driver requires. The `LLVMFuzzerTestOneInput` function signature was wrong. The input parsing was incorrect.

Qwen 2.5-Coder succeeded because it was trained specifically on code generation tasks. It understood the structural patterns of fuzz drivers from its training data. The model had seen enough examples of the correct form to reproduce it reliably. Gemma 3 27B, while classified as a general-purpose model, also succeeded, likely due to its recent architecture and the breadth and quality of its training data, which included a large proportion of code in its training corpus. This matters more than raw parameter count.

The LoRA fine-tuning results reinforced this insight. A small 1.5 billion parameter model, after adaptation on high-quality OSS-Fuzz drivers, produced outputs matching the quality of much larger models while using markedly fewer resources. Specialized knowledge compensated for reduced parameter count.

These findings challenge a common assumption in AI deployment. Organizations often select the largest model available, assuming a higher parameter count correlates

with better results. Our data suggests this does not hold for specialized code generation tasks. The key factors appear to be training data quality, architectural recency, and sufficient (but not necessarily maximum) model size. A well-chosen model, possibly with domain-specific fine-tuning, can match or exceed larger alternatives at a fraction of the cost.

What about the coverage numbers? The original thesis proposal considered that achieving even 40% coverage while saving hours of expert time would constitute meaningful progress. By this standard, the automated approach succeeded. However, as discussed in Chapter 5, our examination of comparable manually-written fuzz drivers in OSS-Fuzz projects suggests expert-crafted drivers typically achieve 60 to 80% line coverage for similar libraries, because they incorporate domain knowledge about edge cases that LLMs may miss.

This gap matters, but so does the time investment. A skilled security engineer might spend 2 to 8 hours writing and debugging a fuzz driver for a new library. Our automated pipeline generates a usable driver in approximately 30 minutes. Even if the automated driver achieves lower coverage, the time savings are meaningful.

6.1.2 Unexpected Findings: Network Architecture

We anticipated the core technical challenge would be generating high-quality fuzz drivers. We expected difficulties with C++ complexity, API understanding, or coverage optimization. We planned our research around these expected challenges.

Infrastructure integration constituted a major focus of the implementation effort.

When we moved from local experiments to CARIAD's production environment, we identified that integrating with the corporate network architecture required additional design considerations. The CI/CD/CT runners operated in isolated network segments by design, and external API access required explicit network path configuration. Our architecture assumed we could call LLM services from build processes, but the corporate network security policies required a tailored approach.

Resolving this required evaluating several architectural approaches. The existing proxy infrastructure was configured for approved HTTP destinations, and the runner environment was designed without VPN client capabilities, consistent with its security-hardened configuration. Caching model responses was considered but would not support dynamic code generation effectively.

We collaborated with CARIAD's infrastructure team to provision Azure Private Link endpoints. This technology creates a private connection between the corporate network and Azure services, appearing as an internal address rather than external internet. The LLM API becomes reachable without traversing the public internet.

The architecture of this solution is illustrated in Figure 6.1, which shows how the Azure Private Link creates a secure tunnel between the CARIAD internal network and Azure OpenAI services, routing traffic through a private connection rather than the public internet while maintaining security compliance.

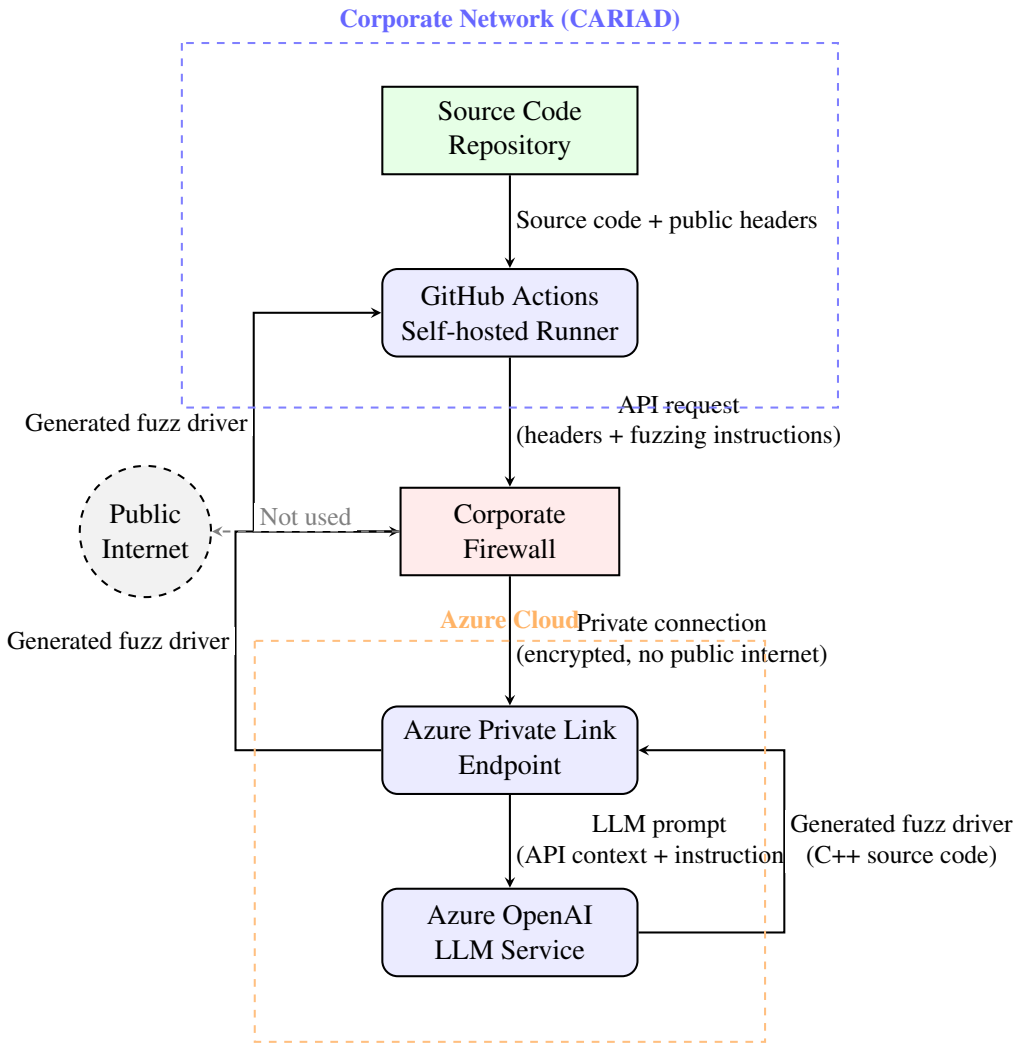

Figure 6.1: Enterprise Network Architecture with Azure Private Link. Processing components are shown in rounded boxes; data stores in rectangular boxes. All edges are labeled with the data exchanged between components. Traffic flows through a private connection and never traverses the public internet.

Setting up Private Link involved organizational coordination, including approval processes, security reviews, configuration testing, and documentation, which are standard steps for enterprise deployments in regulated industries. The technical configuration itself was manageable once approvals were obtained, requiring approximately one month from initial request to operational deployment.

This experience provided an important insight about AI deployment in enterprise environments. The machine learning components are complemented by infrastructure integration requirements. Enterprise environments introduce additional considerations, including network architecture, security compliance, and provisioning workflows, that differ from academic research settings. Anyone planning to deploy LLM-based tools in large organizations should allocate appropriate time for enterprise integration alongside the technical implementation. Based on our experience, organizations should plan for a 2- to 3-month enterprise integration timeline to accommodate standard provisioning and compliance workflows.

6.2 Addressing Research Questions

6.2.1 Primary Research Question Analysis

RQ1: Can Large Language Models generate fuzz drivers for C++ code that compile successfully, execute without errors, and achieve meaningful code coverage?

The answer is affirmative, though several conditions apply.

The experiments demonstrated that specialized code models can generate fuzz drivers meeting these criteria. Results from Chapter 5, Section 5.2.1, show that the best-performing models achieved 43 to 45% line coverage on yaml-cpp.

It is important to contextualize this finding relative to prior work. Previous studies such as TitanFuzz [11] and Fuzz4All [42] have demonstrated that LLMs can generate fuzz targets for Python libraries and deep learning frameworks. The novel contribution of our work is threefold: (1) we demonstrate this capability specifically for C/C++ libraries with libFuzzer-style drivers, which impose stricter structural requirements than Python test cases; (2) we provide a systematic comparison across 14 models showing that model selection has a decisive impact on success; and (3) we validate the approach within an enterprise CI/CD/CT pipeline under real network isolation and security constraints, which no prior work has reported.

However, the qualifications matter. Not all models succeed. General-purpose models, even large ones, frequently did not produce usable output. Model selection matters. In practice, the question "can LLMs do this" should more precisely be "which LLMs can do this, and under what conditions."

While respectable, these coverage numbers likely trail expert human performance. LLM-generated drivers are useful for initial security testing but should not be considered equivalent to carefully crafted manual drivers for critical applications. Their

advantage lies in automation speed, not coverage depth.

RQ2: Does model size determine fuzz driver quality, or can smaller models with domain-specific training match or exceed larger general-purpose models?

The data clearly demonstrates that size alone does not determine quality. However, the picture is more nuanced than a simple "specialized vs. general-purpose" dichotomy.

Evidence from Chapter 5, Section 5.2.1, indicates that the three successful models span both categories: Qwen 2.5-Coder 32B is explicitly code-specialized, while Gemma 3 27B and Phi-4 14B are general-purpose models with strong code capabilities. Meanwhile, several explicitly code-specialized models (CodeLlama 34B, WizardCoder 34B, DeepSeek Coder 33B) failed entirely. Our LoRA fine-tuned 1.5B model achieved 33% faster generation and 55% fewer tokens while producing usable fuzz drivers (see Section 5.3), demonstrating that domain-specific fine-tuning can compensate for reduced model size.

This finding has practical implications. Organizations deploying LLM-based fuzzing should focus on models with recent architectures and high-quality training data rather than selecting solely based on size or code-specialization labels. Fine-tuning smaller models offers a cost-effective alternative when the target task is well-defined.

RQ3: Can LLM-assisted fuzz driver generation be integrated into secure enterprise CI/CD/CT pipelines while meeting performance, cost, and security requirements?

Integration is feasible, but requires approximately one month of infrastructure coordination (security review, Azure Private Link provisioning, DNS configuration, and network testing).

The performance requirement was met. As defined in Chapter 4 (Section 4.4), the performance requirements specified that driver generation must complete within the CI/CD/CT feedback window (targeted at 15 to 30 minutes, with up to 45 minutes acceptable for complex libraries) and that fuzzing execution must not block the main build pipeline. Driver generation completed within 10 to 43 minutes depending on the model (see Table 5.1), with the best models (Qwen 2.5-Coder 32B, Gemma 3 27B) completing in approximately 33 minutes. While this slightly exceeds the 30-minute target for complex libraries, it remains within the acceptable range for asynchronous pipeline execution. Fuzzing execution runs asynchronously without blocking developer feedback.

The cost requirement was met. As detailed in Chapter 5, Section 5.4, enterprise deployment costs (€74 to €1,452 annually) represent less than 2% of a single security engineer's salary, well within the €2,000 annual budget cap defined in Section 4.4.

The security aspect required the most thorough engineering effort. Network isolation policies initially required an alternative deployment approach. The Azure Private Link solution addressed this, but required coordination with infrastructure teams. These requirements are achievable, but organizations should plan integration timelines appropriate to their specific security policies.

6.2.2 Secondary Research Questions Analysis

Beyond the primary questions, our work revealed answers to questions we did not explicitly ask at the start.

What types of libraries are best suited for LLM-generated fuzz drivers?

Experiments across multiple repositories showed that parsing libraries with clear documentation produced better drivers. Libraries with extensive header comments, consistent function signatures, and simple data types yielded higher success rates. Complex APIs with heavy template usage, deeply nested callback patterns, or implicit preconditions produced more failures.

yaml-cpp fell in the middle range. Its parsing APIs are reasonably well documented, but the YAML specification itself introduces complexity. The cross-library results from Chapter 5 Section 5.2.3 revealed additional patterns. jsoncons achieved the highest coverage (45.64%), while the fmt library showed that smaller models can outperform larger ones when the API surface is simpler. Libraries with mathematical interfaces (glm) or configuration-heavy APIs (spdlog) consistently produced 0% coverage, suggesting these patterns remain challenging for current LLMs.

How do generated drivers compare to manually written ones in bug finding capability?

We did not run extended fuzzing campaigns due to time constraints, so our evidence on this question is limited. The coverage data suggests that LLM-generated drivers explore a reasonable subset of code paths. Whether they find the same bugs as expert drivers remains an open question.

Anecdotally, during our experiments, one generated driver did trigger an assertion failure in yaml-cpp that we had not previously known about. This was a minor issue rather than a security vulnerability, but it demonstrates that automated drivers can find real problems.

What failure modes are most common in generated drivers?

Based on our observations during experiments, failures fell into several categories:

1. Structural errors (wrong function signature, missing entry point)
2. Type mismatches (incorrect parameter types, missing casts)
3. Missing includes or dependencies
4. Runtime crashes from invalid API usage

Structural errors were the most frequent issue. The models often generated code that resembled fuzz drivers but lacked the correct `LLVMFuzzerTestOneInput` function signature or proper libFuzzer entry point structure.

The structural errors were often recoverable through prompt refinement. Adding explicit examples of correct fuzz driver structure to the prompt reduced these failures in follow-up experiments.

6.3 Limitations and Constraints

6.3.1 Technical Limitations

Our work has several technical limitations that affect how the results should be interpreted.

Model selection scope. We evaluated 14 open-source LLMs (plus one additional reasoning-focused model, DeepSeek R1, outside the main benchmark), but this represents a small fraction of available models. New models appear monthly, including more advanced versions of GPT-4 [34] and other commercial offerings we could not test due to cost or licensing considerations. Some models we did not test might perform better or worse than our selections. We chose models based on availability, documentation, and community adoption rather than exhaustive search.

Single target language. All experiments focused on C++ libraries. Results might differ for other languages. C has similar challenges, but Python or Rust might show different patterns due to different memory safety characteristics and training data distribution.

Limited vulnerability discovery validation. We measured code coverage as a proxy for fuzzing effectiveness, but coverage does not guarantee bug finding. A driver achieving 45% coverage might miss critical code paths where vulnerabilities exist. We did not run extended fuzzing campaigns to measure actual vulnerability discovery rates.

Hardware constraints. Our local experiments used a single workstation with limited GPU memory. This meant we could not run the largest models at full precision. Quantization may have affected performance for some models, though we attempted to use recommended settings.

Time constraints on fuzzing runs. Each driver was executed for a limited time during evaluation. Longer fuzzing campaigns might reveal different coverage patterns as the fuzzer explores more deeply. Our snapshot measurements capture initial effectiveness but not long-term behavior.

6.3.2 Methodological Boundaries

Beyond technical limitations, our methodology has boundaries that constrain generalization.

Single organization context. The enterprise integration work happened at CARIAD. Their network architecture, security policies, and tooling differ from other organizations. The specific solutions we developed (Azure Private Link, self-hosted runners) may not transfer directly to other environments.

Limited manual driver comparison. We compared against OSS-Fuzz drivers where available, but did not commission expert written drivers specifically for comparison. Recent research on parameter-efficient fine-tuning [19] suggests that such

techniques can improve parameter efficiency by a large margin, but their direct impact on fuzz driver quality remains underexplored. The human baseline is somewhat indirect.

No user study. We did not evaluate how developers interact with generated drivers in practice. Do they trust the output? Do they modify it before use? These human factors affect practical deployment but were outside our scope.

Reproducibility challenges. LLM outputs are stochastic. Running the same prompt multiple times produces different code. We report averages across multiple generations (typically 5 runs per configuration as described in Chapter 5), but exact reproduction of our results requires matching random seeds and model versions.

These limitations do not invalidate our findings, but they define the scope within which the results apply. Organizations considering similar deployments should evaluate their specific context rather than assuming identical outcomes.

6.4 Implications for Practice

6.4.1 Automotive Industry Impact

The automotive industry faces a particular combination of regulatory, complexity, and resource pressures that makes LLM-assisted fuzzing attractive.

Regulatory requirements are tightening. UNECE Regulation 155 [39] mandates cybersecurity management systems for vehicle manufacturers. ISO/SAE 21434 [23] requires systematic security engineering throughout the vehicle lifecycle. Fuzzing provides documented evidence of security testing that regulators recognize.

Software complexity continues to grow. Modern vehicles contain millions of lines of code across networked ECUs. Manual security review cannot scale to this volume.

The cost analysis from Chapter 5, Section 5.4, shows that even at the enterprise usage level, annual costs remain under €1,500. We recommend a phased adoption approach: start with non safety-critical subsystems like infotainment or connectivity features, build confidence and collect metrics, then gradually expand to more critical systems as the approach proves reliable.

6.4.2 Enterprise CI/CD/CT Deployment Patterns

Our integration experience at CARIAD (detailed in Chapter 4) illustrates deployment patterns applicable to other enterprises. We identify four main approaches for integrating LLM services into security-constrained environments:

Private deployment of models. Running models on internal infrastructure avoids external network traffic entirely. This works for smaller models but requires dedicated compute resources. Organizations must weigh infrastructure costs against API costs.

Cloud provider private connectivity. Services like Azure Private Link [29], AWS PrivateLink, and Google Cloud Private Service Connect create private paths to cloud services. These maintain security boundaries while enabling access, provided the cloud provider itself is trusted. As discussed in Chapter 4, our architecture relies on contractual trust (Microsoft Enterprise Agreement, SOC 2 Type II, ISO 27001 certifications) rather than technical guarantees against a compromised provider. For organizations that cannot accept this trust assumption, on-premises model deployment (as evaluated in Phase 1) remains the alternative, though at higher infrastructure cost. Configuration requires coordination between cloud and enterprise network teams.

Edge deployment. Running models on local hardware avoids network dependencies entirely, suitable for latency-sensitive applications with infrequent model updates.

Hybrid approaches. Combining private deployment for sensitive operations with cloud APIs for less sensitive ones provides flexibility at the cost of additional complexity.

6.5 Future Research Directions

Our work opens several directions for future research.

Extended fuzzing campaigns. We measured initial code coverage but did not run campaigns long enough to measure vulnerability discovery. Future work should execute fuzzing for hours or days, tracking crash discovery over time. This would establish whether LLM-generated drivers find bugs at rates comparable to manual drivers.

Multi-model ensembles. Our experiments evaluated models individually. Combining outputs from multiple models might improve quality. One model could generate initial code while another reviews and corrects it. The best combination strategy remains unexplored.

Active learning for driver improvement. When a generated driver fails to compile or crashes at runtime, the error messages contain useful information. Future systems could feed this information back to the LLM for iterative improvement. This closed-loop approach might achieve higher success rates than single-shot generation.

Cross-language generalization. We focused on C++ libraries. Extending to other languages relevant for automotive (C, Rust, Python) would broaden applicability. Different languages likely require different prompt strategies and model selections.

Integration with other testing techniques. LLM-generated fuzz drivers could complement symbolic execution, property-based testing, or formal verification. Understanding how to combine these approaches effectively deserves investigation.

Human-in-the-loop workflows. Rather than fully automated generation, interactive systems could present generated drivers to developers for review and modification. Understanding the right balance between automation and human oversight

would improve practical deployment.

Benchmark development. The field lacks standardized benchmarks for evaluating LLM-based fuzz driver generation. Creating benchmark suites with known ground truth would enable more rigorous comparison across approaches.

Fine-tuning data curation. Our LoRA experiments used OSS-Fuzz drivers as training data. Systematic study of what makes good fine-tuning examples, and how much data is needed, would guide practical deployment.

Security analysis of generated code. LLM-generated code could contain vulnerabilities or backdoors. Analyzing the security properties of generated drivers, beyond their fuzzing effectiveness, matters for deployment in security-sensitive contexts.

6.6 Contributions

This thesis makes the following contributions:

1. **Empirical model evaluation:** A systematic comparison of 14 LLMs for C/C++ fuzz driver generation, establishing that model architecture and training quality matter more than size or specialization label (Chapter 5, Sections 5.2.1–5.2.2).

2. **Fine-tuning for efficiency:** Demonstration that LoRA fine-tuning of a 1.5B model achieves 33% faster generation and 55% fewer tokens while maintaining driver quality (Chapter 5, Section 5.3).

3. **Enterprise integration architecture:** A documented end-to-end architecture for deploying LLM-assisted fuzzing in secure enterprise CI/CD/CT pipelines using Azure Private Link (Chapter 4, Section 4.4).

4. **Economic analysis:** A cost framework showing enterprise deployment costs remain below €1,500 annually (Chapter 5, Section 5.4).

6.7 Conclusion

This work began with a direct question, can Large Language Models automate fuzz driver generation for automotive software? After research, implementation, and deployment effort at CARIAD spanning May through September 2025, the answer is a qualified yes.

Not all models work for this task. General-purpose models without sufficient code training, even large ones, frequently did not produce usable output. But neither did all code-specialized models: CodeLlama and WizardCoder, both explicitly code-focused, also failed. The successful models (Qwen 2.5-Coder, Gemma 3, Phi-4) share recent architectures and high-quality training data. Model selection is a key design decision with measurable impact on output quality.

Fine-tuning small models proved more effective than expected. Our LoRA-adapted 1.5B model achieved measurable efficiency gains (detailed in Chapter 5) while producing usable fuzz drivers. For organizations with compute constraints or cost sensitivity, this approach deserves consideration.

An important finding was the role of infrastructure. We expected the primary focus to be generating good code. In practice, a large portion of the project timeline was spent on deploying our solution within the corporate network architecture. Initial approaches including proxy configuration, boundary client integration, and container networking were iteratively refined to align with the enterprise network architecture. The solution ultimately required organizational support for Azure Private Link architecture.

Economically, adoption is well supported. As shown in Chapter 5, LLM-assisted fuzzing costs €74 to €1,452 annually, representing less than 2% of a single security engineer's salary and well within typical automotive tooling budgets. The value comes from scaling security testing to match code production rates that manual approaches cannot achieve.

Looking forward, LLM-assisted security testing will likely become standard practice in automotive software development. The technology works well enough today for practical deployment. It will improve as models advance and deployment patterns mature. Organizations that develop expertise now will be better positioned as the technology evolves.

For CARIAD specifically, this work establishes a foundation for expanded deployment. The architecture can accommodate additional target libraries. The cost model supports scaling. The infrastructure patterns can replicate across teams.

More broadly, our findings extend beyond fuzzing. Automotive software development faces a core scaling problem. Code complexity grows faster than team sizes. AI-assisted tools offer a path to maintain quality at scale. Fuzz driver generation is one application of this principle. Others will follow.

One key insight from this research is that infrastructure integration complexity can exceed the technical challenges of the core technology. While specific components like Azure Private Link required approximately one month for deployment, the complete enterprise integration followed standard organizational workflows for evaluation, approval, infrastructure provisioning, testing, and documentation. Organizations planning similar deployments should allocate 2 to 3 months for the full integration process and plan time commensurate with their security policy complexity.

A Appendix

A.1 Evaluated Model Summary

Table A.1 provides a complete overview of all LLM models evaluated during Phase 1 of this research, including their parameter counts, quantization levels, architecture types, and primary results on yaml-cpp.

Table A.1: Complete List of Evaluated Models

Model	Architecture	Parameters	Quantization	yaml-cpp Coverage	Status
Qwen 2.5-Coder 32B	Decoder-only	32B	Q4_K_M	43.08%	Functional
Gemma 3 27B	Decoder-only	27B	Q4_K_M	45.06%	Functional
Phi-4 14B	Decoder-only	14B	Q4_K_M	34.26%	Functional
Qwen 2.5-Coder 14B	Decoder-only	14B	Q5_K_M	0%	Compile failure
Qwen 2.5-Coder 7B	Decoder-only	7B	Q5_K_M	0%	Compile failure
Qwen 2.5-Coder 1.5B	Decoder-only	1.5B	FP16	0%	Compile failure
DeepSeek Coder 33B	Decoder-only	33B	Q4_K_M	0%	Compile failure
DeepSeek Coder V2 Lite	MoE	16B	Q4_K_M	0%	Compile failure
CodeLlama 34B Instruct	Decoder-only	34B	Q4_K_M	0%	Compile failure
WizardCoder 34B	Decoder-only	34B	Q4_K_M	0%	Compile failure
Yi 34B	Decoder-only	34B	Q4_K_M	0%	Invalid API usage
Gemma 3 9B	Decoder-only	9B	Q4_K_M	0%	Compile failure
Gemma 3 4B	Decoder-only	4B	Q4_K_M	0%	Compile failure
Mixtral 46.7B	MoE	46.7B	Q4_K_M	N/A	Hardware limit

A.2 LoRA Fine-Tuning Script

The following script shows the complete LoRA fine-tuning configuration used in Phase 2 (Section 4.3).

```
from transformers import AutoModelForCausalLM, AutoTokenizer,
    TrainingArguments
from peft import LoraConfig, get_peft_model
from datasets import load_dataset

# Model and tokenizer
MODEL_NAME = "Qwen/Qwen2.5-Coder-1.5B"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForCausalLM.from_pretrained(MODEL_NAME,
    torch_dtype="float16", device_map="auto")
```

```python

# LoRA configuration
lora_config = LoraConfig(
    r=16,                    # Rank: controls adaptation complexity
    lora_alpha=32,           # Scaling factor
    lora_dropout=0.1,        # Dropout to prevent overfitting
    target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
    task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora_config)

# Training arguments
training_args = TrainingArguments(
    output_dir="./fuzz-lora-checkpoints",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-5,
    lr_scheduler_type="cosine",
    warmup_steps=100,
    fp16=True,
    logging_steps=10,
    save_strategy="epoch",
    evaluation_strategy="epoch",
)

# Dataset format: {"input": "<headers>", "output": "<fuzz_driver>"}
dataset = load_dataset("json", data_files="ossfuzz_drivers_709.jsonl")
```

Listing A.1: LoRA Fine-Tuning Script for Fuzz Driver Generation

A.3 Sample Generated Fuzz Driver

Listing A.2 shows a representative fuzz driver generated by Qwen 2.5-Coder 32B for the yaml-cpp library. This driver achieved 43% line coverage during our evaluation (Chapter 5).

```cpp
#include <cstdint>
#include <cstddef>
#include <string>
#include "yaml-cpp/yaml.h"

extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    if (size == 0) return 0;
```

```
8
9     // Convert fuzzer input to string
10    std::string input(reinterpret_cast<const char*>(data), size)
      ;
11
12    try {
13        // Parse YAML from the fuzzer-provided input
14        YAML::Node doc = YAML::Load(input);
15
16        // Exercise different node types
17        if (doc.IsMap()) {
18            for (auto it = doc.begin(); it != doc.end(); ++it) {
19                std::string key = it->first.as<std::string>();
20                if (it->second.IsScalar()) {
21                    std::string val = it->second.as<std::string
      >();
22                }
23            }
24        } else if (doc.IsSequence()) {
25            for (size_t i = 0; i < doc.size(); ++i) {
26                if (doc[i].IsScalar()) {
27                    std::string val = doc[i].as<std::string>();
28                }
29            }
30        }
31    } catch (const YAML::Exception&) {
32        // Expected for malformed input
33    } catch (const std::exception&) {
34        // Catch other standard exceptions
35    }
36
37    return 0;
38 }
```

Listing A.2: Sample LLM-Generated Fuzz Driver for yaml-cpp

A.4 Pipeline Configuration

Listing A.3 shows the complete GitHub Actions workflow configuration for CI/CD/CT integration with Azure OpenAI, as discussed in Chapter 4, Section 4.4. The workflow diagram in Figure 4.2 visualizes this pipeline.

```
1 name: fuzz
2 env:
3   PROJ_NAME: "<project-name>"
4   VERSION: "0.1"
5   ARTIFACTORY_CORPUS_URL: "https://<corporate-artifactory>.
     example.com/..."
```

```
6  jobs:
7    fuzz:
8      runs-on: [<self-hosted-runner>]
9      env:
10       JFROG_API_KEY: ${{ secrets.JFROG_API_KEY }}
11       ARTIFACTORY_USER: ${{ secrets.ARTIFACTORY_USER }}
12       PROJECT_FOLDER: "/project/"
13       CIFUZZ_LLM_API_URL: "https://<endpoint>.openai.azure.com"
14       CIFUZZ_LLM_AZURE_DEPLOYMENT_NAME: "gpt-4o"
15       CIFUZZ_LLM_MAX_TOKENS: "40000"
16       CIFUZZ_LLM_API_TOKEN: ${{ secrets.CIFUZZ_LLM_API_TOKEN }}
17     steps:
18       - uses: actions/checkout@v3
19       - name: Login to JFrog
20         run: buildah login -u $ARTIFACTORY_USER -p
     $JFROG_API_KEY ...
21       - name: Download previous corpus
22         run: |
23           # Download corpus archive if it exists in Artifactory
24           curl -u "$ARTIFACTORY_USER:$JFROG_API_KEY" ... | tar -
     xz
25       - name: Pull cifuzz container
26         run: |
27           containerid=$(buildah from .../cifuzz:${VERSION})
28           echo "containerid=$containerid" >> $GITHUB_ENV
29       - name: Build project
30         run: |
31           buildah run -v $(pwd):/$PROJECT_FOLDER $containerid \
32             /bin/bash -c "cd /project && mkdir build && cd build
      && cmake .. && make"
33       - name: Generate fuzz driver via LLM
34         run: |
35           buildah run --env CIFUZZ_LLM_API_URL ... \
36             /bin/bash -c "cd /project/build && cifuzz spark
     parser_target"
37       - name: Rebuild with generated tests
38         run: buildah run ... "cd /project/build && make"
39       - name: Run fuzzing (60s, ASan+UBSan)
40         run: |
41           buildah run ... "cifuzz run parser_target \
42             --libfuzzer-arg=-max_total_time=60"
43       - name: Upload updated corpus
44         run: |
45           tar -czf ${PROJ_NAME}_corpus.gz .cifuzz-corpus
46           curl -T ${PROJ_NAME}_corpus.gz ...
```

Listing A.3: Complete GitHub Actions Workflow with Azure OpenAI Integration

A.5 Sample LLM Prompt for Fuzz Driver Generation

Listing A.4 shows a representative prompt sent to the LLM during fuzz driver generation. The prompt combines the extracted API context (abbreviated for space) with the fuzzing instructions that define the required driver structure. This example targets the yaml-cpp library.

```
=== SYSTEM ===
You are a security-focused C++ developer specializing in fuzz testing
with libFuzzer. Generate a fuzz driver for the target API.

=== FUZZING INSTRUCTIONS ===
1. The driver MUST implement:
   extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
2. Consume the byte buffer ‘data‘ of length ‘size‘ as input.
3. Translate bytes into valid API calls to the target library.
4. Do NOT call exit(), abort(), or _exit().
5. Do NOT use static or global state that persists across calls.
6. Catch and handle exceptions; do not let them propagate.
7. Return 0 on success.

=== EXTRACTED API CONTEXT ===
// File: yaml-cpp/yaml.h
namespace YAML {
  class Node {
  public:
    Node();
    Node Load(const std::string& input);
    bool IsMap() const;
    bool IsSequence() const;
    bool IsScalar() const;
    // ... (50+ additional methods) ...
    template<typename T> T as() const;
    iterator begin();
    iterator end();
    std::size_t size() const;
  };
  // ... (additional classes: Parser, Emitter, Exception) ...
}

=== TASK ===
Generate a complete fuzz driver that exercises the YAML::Node API.
Include all necessary #include directives.
Maximize code coverage of the target library.
```

Listing A.4: Representative cifuzz spark Prompt (abbreviated)

References

[1] AFLplusplus Team. *AFL++: afl-fuzz Approach*. 2024. URL: https://aflplus.plus/.

[2] AUTOSAR Development Partnership. *AUTOSAR – AUTomotive Open System ARchitecture*. 2024. URL: https://www.autosar.org/.

[3] *Azure DevOps Documentation*. Microsoft. 2024. URL: https://learn.microsoft.com/en-us/azure/devops/.

[4] Jinze Bai et al. *Qwen Technical Report*. arXiv:2309.16609. 2023. URL: https://arxiv.org/abs/2309.16609.

[5] Manfred Broy. "Challenges in Automotive Software Engineering". In: *ICSE '06*. 2006. URL: https://doi.org/10.1145/1134285.1134292.

[6] *Buildah – A Tool That Facilitates Building OCI Container Images*. Containers, 2024. URL: https://buildah.io/.

[7] Robert N. Charette. "This Car Runs on Code". In: *IEEE Spectrum* (Feb. 2009). URL: https://spectrum.ieee.org/this-car-runs-on-code.

[8] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374. 2021. URL: https://arxiv.org/abs/2107.03374.

[9] Code Intelligence. *cifuzz: Coverage-Guided Fuzzing for C/C++ Projects*. The `spark` module provides LLM-based automated fuzz driver generation via API context extraction. 2024. URL: https://www.code-intelligence.com/cifuzz.

[10] Yinlin Deng et al. *Large Language Models Are Edge-Case Fuzzers: Testing Deep Learning Libraries via FuzzGPT*. arXiv:2304.02014. 2023. URL: https://arxiv.org/abs/2304.02014.

[11] Yinlin Deng et al. *Large Language Models are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models*. arXiv:2212.14834. 2022. URL: https://arxiv.org/abs/2212.14834.

[12] Zhen Yu Ding and Claire Le Goues. *An Empirical Study of OSS-Fuzz Bugs*. arXiv:2103.11518. 2021. URL: https://arxiv.org/abs/2103.11518.

[13] Xinyi Du et al. *Large Language Models for Fuzzing: A Survey*. arXiv:2402.12318. 2024. URL: https://arxiv.org/abs/2402.12318.

[14] Martin Fowler. "Continuous Integration". In: *martinfowler.com* (2006). URL: https://martinfowler.com/articles/continuousIntegration.html.

[15] Patrice Godefroid, Michael Y. Levin, and David Molnar. "Automated Whitebox Fuzz Testing". In: *NDSS*. 2008. URL: https://patricegodefroid.github.io/public_psfiles/ndss2008.pdf.

[16] Google. *OSS-Fuzz-Gen: LLM-powered fuzzing via domain-specific fine-tuning*. Automated fuzz target generation for OSS-Fuzz projects using LLMs. 2024. URL: https://github.com/google/oss-fuzz-gen.

[17] Google. *Syzkaller: Coverage-Guided Kernel Fuzzing*. 2020. URL: https://github.com/google/syzkaller.

[18] Andy Greenberg. "Hackers Remotely Kill a Jeep on the Highway—I Was in It". In: *Wired* (July 2015). FCA subsequently recalled 1.4 million vehicles (NHTSA Campaign 15V-461). URL: https://www.wired.com/2015/07/hackers-remotely-kill-jeep-highway/.

[19] Zeyu Han et al. *Parameter-Efficient Fine-Tuning Methods for Large Language Models: A Survey*. arXiv:2312.12148. 2024. URL: https://arxiv.org/abs/2312.12148.

[20] Edward J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685. 2021. URL: https://arxiv.org/abs/2106.09685.

[21] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010. URL: https://continuousdelivery.com/.

[22] *ISO 26262: Road Vehicles – Functional Safety*. International Organization for Standardization. Geneva, Switzerland, 2018. URL: https://www.iso.org/standard/68383.html.

[23] *ISO/SAE 21434: Road Vehicles – Cybersecurity Engineering*. International Organization for Standardization. Geneva, Switzerland, 2021. URL: https://www.iso.org/standard/70918.html.

[24] Albert Q. Jiang et al. *Mistral 7B*. arXiv:2310.06825. 2023. URL: https://arxiv.org/abs/2310.06825.

[25] Thijs Klooster et al. *Effectiveness and Scalability of Fuzzing Techniques in CI/CD Pipelines*. arXiv:2205.14964. 2022. URL: https://arxiv.org/abs/2205.14964.

[26] Fred Lambert. *Tesla Has Updated Its Cars Over the Air More Than Any Other Automaker*. Electrek. 2019. URL: https://electrek.co/2019/08/01/tesla-car-software-update/.

[27] Ruijie Meng et al. *HyLLfuzz: Large Language Model-Guided Hybrid Fuzzing*. Preprint. Hybrid approach combining LLM-generated seeds with mutation-based fuzzing. 2024.

[28] Microsoft. *Azure OpenAI Service Documentation*. 2024. URL: https://learn.microsoft.com/en-us/azure/ai-services/openai/.

[29] Microsoft. *Azure Private Link Documentation*. 2024. URL: https://learn.microsoft.com/en-us/azure/private-link/.

[30] Barton P. Miller, Louis Fredriksen, and Bryan So. "An Empirical Study of the Reliability of UNIX Utilities". In: *Communications of the ACM* 33.12 (1990), pp. 32–44. URL: https://doi.org/10.1145/96267.96279.

[31] Charlie Miller and Chris Valasek. *Remote Exploitation of an Unaltered Passenger Vehicle*. Black Hat USA. 2015. URL: http://illmatics.com/Remote%5C%20Car%5C%20Hacking.pdf.

[32] Augustus Odena et al. "TensorFuzz: Debugging Neural Networks with Coverage-Guided Fuzzing". In: *ICML*. 2019. URL: https://arxiv.org/abs/1807.10875.

[33] Ollama Team. *Ollama: Run Large Language Models Locally*. 2024. URL: https://ollama.ai/.

[34] OpenAI. *GPT-4 Technical Report*. arXiv:2303.08774. 2023. URL: https://arxiv.org/abs/2303.08774.

[35] Kexin Pei et al. "DeepXplore: Automated Whitebox Testing of Deep Learning Systems". In: *SOSP*. 2017. URL: https://doi.org/10.1145/3132747.3132785.

[36] Gary J Saavedra et al. *A Review of Machine Learning Applications in Fuzzing*. Tech. rep. Sandia National Laboratories, 2019. URL: https://doi.org/10.2172/1529399.

[37] Kostya Serebryany. "Continuous Fuzzing with libFuzzer and AddressSanitizer". In: *IEEE Cybersecurity Development (SecDev)*. 2016. URL: https://llvm.org/docs/LibFuzzer.html.

[38] Hugo Touvron et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. arXiv:2307.09288. 2023. URL: https://arxiv.org/abs/2307.09288.

[39] UNECE. *UN Regulation No. 155: Uniform provisions concerning the approval of vehicles with regards to cyber security and cyber security management system*. Official Journal of the European Union. 2021. URL: https://unece.org/transport/documents/2021/03/standards/un-regulation-no-155.

[40] Bo Wang et al. *A Comprehensive Study on Large Language Models for Mutation Testing*. arXiv:2406.09843. 2024. URL: https://arxiv.org/abs/2406.09843.

[41] Yan Wang, Peng Jia, and Luping Liu. "A systematic review of fuzzing based on machine learning techniques". In: *Applied Sciences* 10.16 (2020). URL: https://doi.org/10.3390/app10165648.

[42] Chunqiu Steven Xia et al. *Fuzz4All: Universal Fuzzing with Large Language Models*. arXiv:2308.04748. 2023. URL: https://arxiv.org/abs/2308.04748.

[43] Chenyuan Yang et al. *WhiteFox: White-Box Compiler Fuzzing Empowered by Large Language Models*. arXiv:2310.15991. 2023. URL: https://arxiv.org/abs/2310.15991.

[44] Tuba Yavuz. *Multi-Pass Targeted Dynamic Symbolic Execution (DESTINA)*. 2024. URL: https://doi.org/10.1145/3597926.3598059.

[45] Cen Zhang et al. *How Effective Are They? Exploring Large Language Model Based Fuzz Driver Generation*. arXiv:2307.12469. 2023. URL: https://arxiv.org/abs/2307.12469.

Erklärung

Ich versichere, dass ich die Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Erlangen,

Morris Darren Babu